

Diskreeteistä tekstiesityksistä SQL-kyselyiden generointiin: kielimallien toimintaperiaatteet ja Text-to-SQL-järjestelmän luotettavuuden arviointi

Omar Mayani

Pro gradu -tutkielma
Marraskuu 2026

MATEMATIIKAN JA TILASTOTIETEEN LAITOS

Tarkastajat:
Prof. Ion Petre
Markus Ylikojola

Turun yliopiston laatujärjestelmän mukaisesti tämän julkaisun alkuperäisyys on tarkastettu Turnitin OriginalityCheck-järjestelmällä

TURUN YLIOPISTO, Matematiikan ja tilastotieteen laitos

Pro gradu -tutkielma
Pääaine: Matematiikka
Tekijä: Omar Mayani
Otsikko: Diskreeteistä tekstiesityksistä SQL-kyselyiden generointiin: kielimallien toimintaperiaatteet ja Text-to-SQL-järjestelmän luotettavuuden arviointi
Ohjaaja: Prof. Ion Petre
Sivumäärä: 58 sivua
Aika: Marraskuu 2026

Tiivistelmä

Tässä tutkielmassa arvioidaan kielimallipohjaisen Text-to-SQL-järjestelmän kykyä muodostaa käyttäjän tiedontarvetta vastaavia SQL-kyselyitä sekä tulkinnan johdonmukaisuutta kysymysten toistoissa ja sanamuodon muutoksissa.

Teoreettisessa osassa jäsennetään se matemaattinen ja arkkitehtuurinen jatkumo, jolla diskreeteistä tekstiesityksistä, optimointimenetelmistä ja itsehuomiomekanismista rakentuu symbolista koodia tuottava autoregressiivinen todennäköisyysmalli. Teoria tarjoaa käsitteellisen perustan kielimallin toiminnalle ja havaitun vaihtelun ymmärtämiselle.

Empiirinen osa on eksploratiivinen pilottitutkimus Snowflaken Cortex Analyst -järjestelmästä, joka hyödyntää semanttista näkymää puhelinkeskusdataa sisältävässä tietokantaympäristössä.

Tutkimukseen valittiin viisi tiedontarvetta. Kullekkin tiedontarpeelle muodostettiin viisi kysymystä ja kukin näistä esitettiin järjestelmälle kymmenen kertaa. Yhteensä 250 vastausta arvioitiin manuaalisesti tuotettujen SQL-kyselyiden perusteella. Tiedontarpeen tulkinnan vaihtelu erotettiin saman tulkinnan vaihtoehtoisista SQL-toteutuksista.

Suorituksista 173 tuotti oikean, 18 melkein oikean ja 19 väärän SQL-kyselyn. SQL-kysely jäi muodostamatta 40 suorituksessa. Selkeät kysymykset ja semanttisen näkymän valmiiden metriikoiden tukemat tehtävät tuottivat vakaita tulkintoja. Ongelmia ilmeni erityisesti implisiittisten taustaoletusten huomioimisessa, suodatuksissa ja duplikaattien käsittelyssä. Sanamuodon muutoksiin liittyi myös tulkintaeroja.

Tulosten perusteella tulkinnan johdonmukaisuus ei takaa oikeellisuutta, eikä SQL-toteutusten vaihtelu itsessään osoita epäluotettavuutta. Järjestelmä tuotti tiedontarvetta vastaavia kyselyitä valituissa analyysitehtävissä, mutta oikeellisuus ei säilynyt kaikissa kysymysmuotoiluissa ja suorituksissa. Havainnot puoltavat kyselyiden oikeellisuuden varmistamista tutkitussa sovellusympäristössä. Tulokset ovat tapauskohtaisia eivätkä muodosta kattavaa arviota järjestelmän yleisestä suorituskyvystä.

Avainsanat: kielimallit, Text-to-SQL, Cortex Analyst, Transformer, semanttinen näkymä, luotettavuus, autoregressiivinen mallinnus



Teknologiateollisuus

Tämän pro gradu -tutkielman tutkimustyötä on tuettu Teknologiateollisuuden 100-vuotissäätiön myöntämällä apurahalla.

MERKINTÄTAVOISTA

Tässä opinnäytetyössä käytetään seuraavia merkintöjä ja termejä.

$\mathbb{R}$	reaalilukujen joukko
$\mathbb{N}$	luonnollisten lukujen joukko $\{1, 2, 3, \dots\}$
$A^\top$	matriisin tai vektorin A transpoosi
$\odot$	Hadamard-tulo (alkioittainen kertolasku)
$[A \,\|\| \, B]$	vektorien tai matriisien vaakasuuntainen ketjuttaminen
$\|S\|$	joukon tai sekvenssin S alkioiden lukumäärä
$\mathbf{1}\{\cdot\}$	indikaattorifunktio
$\mathcal{V} = \{v_1, \dots, v_{\|\mathcal{V}\|}\}$	sanasto (kaikkien käytettävien tokenien joukko)
$w_t \in \mathcal{V}$	korpuksen ajanhetken t tokeni
$\mathcal{W} = (w_1, \dots, w_{\|\mathcal{W}\|})$	korpus (havaittu tokenijono)
$x(v_j) \in \{0,1\}^{1\times\|\mathcal{V}\|}$	tokenin v_j one-hot-esitys (rivivektori)
$e(v_j) \in \mathbb{R}^{1\times d}$	tokenin v_j tiheä upotusvektori (rivivektori)
$E \in \mathbb{R}^{\|\mathcal{V}\|\times d}$	tokenien upotusmatriisi
$H \in \mathbb{R}^{T\times d}$	esitysmatriisi (jonon pituus T, upotusdimensio d)
$\ell \in \mathbb{R}^{1\times\|\mathcal{V}\|}$	normalisoimaton pistemäärävektori (logittivektori)
$J(\theta)$	häviöfunktio
θ	mallin opittavien parametrien joukko
$\Delta(\mathcal{V})$	sanaston $\mathcal{V}$ yli määriteltyjen todennäköisyysjakaumien joukko

Sisällys

1 Johdanto **4**

2 Diskreetit tekstiesitykset ja klassinen kielimallinnus **7**

3 Jatkuvat esitykset ja neuroverkkopohjainen kielimallinnus **10**
3.1 Upotusvektorit 10
3.2 Upotusvektorien oppiminen Word2Vec-menetelmällä 11
3.3 N-gram-malli optimointiongelmana 13
3.4 Monikerroksinen perseptroni seuraavan tokenin ennustajana 15

4 Gradienttimenetelmä **18**
4.1 Häiriöalttius ja toisen kertaluvun geometria 21
4.2 Adaptiiviset optimointimenetelmät 22

5 Dekooderipohjainen Transformer-arkkitehtuuri suurille kielimalleille **27**
5.1 Itsehuomio 27
5.2 Positiokoodaus 30
5.3 Dekooderikerros 31
5.4 Dekooderikerroksista seuraavan tokenin jakaumaan 33
5.5 Autoregressiivinen koulutus ja generointi 34
5.6 Kielimalli Text-to-SQL-tehtävissä 36

6 Metodologia **40**
6.1 Tutkimusasetelma 40
6.2 Tutkimusaineisto 40
6.3 Cortex Analyst ja semanttinen näkymä 40
6.4 Arviointiperusteet 42

7 Kokeet **47**

8 Tulokset **50**

9 Rajoitukset **55**

10 Johtopäätökset **57**

1 Johdanto

Kielimallit ovat luonnollisen kielen käsittelyn malleja, joiden avulla voidaan kuvata tekstin tilastollista rakennetta tai tuottaa uutta tekstiä. Autoregressiivisessa kielimallinnuksessa malli ennustaa seuraavaa sanaa (tai tokenia eli tekstin käsittelyyksikköä) aiemman kontekstin perusteella [17, 3]. Tekstiä voidaan generoida valitsemalla mallin ennustamasta jakaumasta seuraava token, liittämällä se kontekstiin ja toistamalla menettelyä päivitetyllä kontekstilla.

Kielimallien kehityksessä keskeinen muutos on ollut siirtyminen diskreeteistä tekstiesityksistä opittuihin jatkuviin esityksiin [23] sekä esiintymismääriin perustuvista malleista neuroverkkopohjaiseen mallinnukseen [45, 34]. Upotusvektoreiden avulla tokenit esitetään vektoriavaruudessa, jossa niiden väliset suhteet voidaan oppia aineistosta. Samalla neuroverkon parametreja optimoidaan tuottamaan opetusaineistoa vastaavia jakaumia. Nykyaikaisissa dekooderipohjaisissa Transformer-malleissa kontekstin käsittely perustuu itsehuomioon [43], jonka avulla tokenien esityksiä muodostetaan suhteessa muihin kontekstin tokeneihin. Kielimallien toiminnan ymmärtäminen edellyttää siten tekstiesitysten, todennäköisyysmallinnuksen, optimoinnin ja neuroverkkoarkkitehtuurin tarkastelua yhtenä kokonaisuutena.

Suurten kielimallien sovellukset eivät rajoitu vapaamuotoisen tekstin tuottamiseen. Kielimalleja voidaan liittää järjestelmiin, joissa osa niiden tuottamista tokeneista tulkitaan työkalukutsuina [35, 48]. Tällöin kielimallia ympäröivä ohjelmisto voi suorittaa laskentaa, tehdä verkkohakuja tai käsitellä tietokantakyselyitä. Yksi käytännössä merkittävä sovelluskohde on luonnollisen kielen muuntaminen SQL-kyselyiksi eli Text-to-SQL [21, 10]. Sen tavoitteena on mahdollistaa tietokantojen hyödyntäminen ilman, että käyttäjän tarvitsee itse kirjoittaa SQL-kyselyitä tai tuntea tietokannan rakennetta yksityiskohtaisesti.

Text-to-SQL-järjestelmän hyödyllisyys riippuu kuitenkin siitä, kuinka luotettavasti se kykenee yhdistämään käyttäjän tiedontarpeen tietokannan rakenteeseen ja muodostamaan tarkoitusta vastaavan kyselyn [49, 19]. Syntaktisesti kelvollinen SQL-kysely ei välttämättä vastaa käyttäjän tarkoittamaa kysymystä. Luotettavuuden kannalta on myös olennaista, säilyykö järjestelmän tulkinta johdonmukaisena saman kysymyksen toistoissa ja silloin, kun kysymyksen esitystapaa muutetaan sen merkitystä muuttamatta. Nämä kysymykset motivoivat kielimallipohjaisten järjestelmien empiiristä arviointia.

Tämän tutkielman keskeisenä tutkimuskysymyksenä on selvittää, miten luotettavasti semanttista näkymää hyödyntävä kielimallipohjainen Text-to-SQL-järjestelmä muodostaa käyttäjän tiedontarvetta vastaavia SQL-kyselyitä ja miten johdonmukaisena sen tulkinta säilyy saman kysymyksen toistoissa sekä kielellisissä variaatioissa, joiden tavoitteena on säilyttää kysymyksen merkitys.

Tutkielman kontribuutio Text-to-SQL-tutkimukselle muodostuu kahdesta toisiaan tukevasta näkökulmasta. Teoreettisella tasolla työn tavoitteena on jäsentää se matemaattinen ja arkkitehtuurinen jatkumo, jolla diskreeteistä esityksistä, optimointimenetelmistä ja itsehuomiosta rakentuu koodia tuottava todennäköisyysmalli. Empiirisellä tasolla tutkielma tuottaa tapauskohtaista tietoa kaupallisen Text-to-SQL-järjestelmän käyttäytymisestä aidossa puhelinkeskusympäristössä.

Text-to-SQL-järjestelmän käyttäytymisestä aidossa puhelinkeskusympäristössä.

Tarkastelussa arvioidaan tuotettujen SQL-kyselyiden vastaavuutta käyttäjän tiedontarpeeseen, tulkintojen johdonmukaisuutta kysymysten toistoissa ja sanamuodon muutoksissa sekä tulkinnan vaihtelusta erillistä toteutusvaihtelua.

Tutkielman empiirinen osa toteutetaan eksploratiivisena pilottitutkimuksena ja tapaustutkimuksena, jossa tarkastellaan Snowflaken Cortex Analyst -järjestelmää [39]. Järjestelmä muodostaa luonnollisen kielen kysymyksistä SQL-kyselyitä hyödyntäen tietokannan rakennetta ja käsitteitä kuvaavaa semanttista näkymää. Aineistona on puhelinkeskusdataa sisältävä tietokanta, joka koostuu yhdestä keskusteludataa sisältävästä päätaulusta ja kolmesta sitä täydentävästä taulusta.

Tarkastelu rajataan viiteen tarkoituksenmukaisesti valittuun tiedontarpeeseen, niiden toistuviin suorituksiin ja vaihtoehtoisiin kysymysmuotoiluihin. Kysymykset valittiin sovellusympäristön analyysitarpeiden ja semanttisen näkymän rakenteen perusteella siten, että mukana on sekä valmiiden metriikoiden tukemia että useiden kenttien ja ehtojen yhdistämistä edellyttäviä tehtäviä. Toistot ja sanamuotovariaatiot mahdollistavat näiden tiedontarpeiden sisäisen vaihtelun tarkastelun, mutta eivät laajenna tehtäväkohtaista kattavuutta. Tavoitteena on tuottaa tapauskohtaisia havaintoja ja lähtökohtia jatkotutkimukselle, ei kattavaa arviota järjestelmän yleisestä suorituskyvystä.

Teoreettinen tarkastelu tarjoaa käsitteellisen perustan kielimallipohjaisen kyselymuodostuksen ymmärtämiselle, mutta sitä ei tule tulkita Cortex Analystin sisäisen toteutuksen kuvaukseksi. Kaupallisten kielimallijärjestelmien tarkat toteutustiedot eivät yleensä ole kokonaisuudessaan julkisia, joten tapaustutkimuksessa järjestelmää arvioidaan sen tuottamien SQL-kyselyiden perusteella. Tarkastelun kohteena on näin ollen järjestelmän havaittava toiminta, ei yksittäisten mallikomponenttien vaikutusten eristäminen.

Arviointi kohdistuu tuotettuihin SQL-kyselyihin eikä niiden suorittamisen palauttamiin tulosjoukkoihin. Lisäksi tarkastellaan vastauksia, joissa järjestelmä ei tuota SQL-kyselyä. Rajaus on perusteltu aineiston luottamuksellisuudella sekä sillä, että virheellisen kyselyn vaikutus tulosjoukkoon riippuu myös tietokannan sisällöstä.

Erilaiset SQL-kyselyt voivat vastata samaan tiedontarpeeseen, vaikka niiden palauttamien tietojen esitysmuoto tai laajuus poikkeaisi toisistaan. Arviointi ei siten edellytä kyselyiden täydellistä semanttista ekvivalenssia, vaan siinä tarkastellaan, sisältäisikö kyselyn tuottama vastaus käyttäjän tarvitseman tiedon sovellusalueella perustelluin rajauksin ja taustaoletuksin.

Tiedontarpeen tulkinnan vaihtelu erotetaan SQL-kyselyiden toteutusvaihtelusta, eikä toteutusvaihtelu itsessään osoita virhettä tai tulkinnan epäjohdonmukaisuutta. Tulokset kuvaavat järjestelmän toimintaa valitussa tutkimusasetelmassa, eivät sen yleistä suorituskykyä kaikissa Text-to-SQL-tehtävissä.

Tutkielma jakautuu teoriaosaan ja empiiriseen osaan. Luvuissa 2–5 rakennetaan kielimallien matemaattinen ja arkkitehtuurinen perusta diskreeteistä esityksistä moderniin Transformer-malliin ja optimointiin, mikä luo pohjan ymmärtää todennäköisyyspohjaisen generoinnin ominaisuuksia symbolisessa päättelyssä. Luvussa 6 esitellään Cortex Analystiin pohjautuva tutkimusasetelma, arviointikriteerit ja semanttisen mallin rooli tutkimuskysymyksen operationalisoinnissa. Luvuissa 7–8 kuvataan kokeet ja analysoidaan tulokset, minkä jälkeen luvuissa 9–10 pohditaan työn rajoituksia sekä esitetään johtopäätökset.

Tämän tutkielman kirjoittamisessa on hyödynnetty GPT-, Claude- ja Gemini-kielimalleja kielenhuoltoon ja tiedonhakuun.

2 Diskreetit tekstiesitykset ja klassinen kielimallinnus

Kielimallipohjaisessa Text-to-SQL-tehtävässä järjestelmän tavoitteena on muuntaa luonnollisen kielen tiedontarve symboliseksi SQL-kyselyksi. Tämän prosessin matemaattinen perusta nojaa kielen esittämiseen laskennallisessa muodossa.

Klassisessa kielimallinnuksessa tekstiä käsitellään diskreetteinä tokeneina. Olkoon $\mathcal{V} = \{v_1, v_2, \ldots, v_{|\mathcal{V}|}\}$ sanasto eli joukko kaikista mallin käyttämistä tokeneista ja olkoon $\mathcal{W} = (w_1, w_2, \ldots, w_{|\mathcal{W}|})$ korpus eli havaittu tokenijono. Tällöin $w_t \in \mathcal{V}$ merkitsee korpuksen ajanhetken t tokenia. Tokenilla voidaan tarkoittaa esimerkiksi sanaa, osasanaa, merkkiä tai muuta diskreettiä tekstiyksikköä riippuen käytetystä tokenisointimenetelmästä [17].

Diskreetit tokenit esitetään numeerisessa muodossa, jotta niitä voidaan hyödyntää tilastollisissa malleissa ja neuroverkoissa. Yksinkertaisin ja yleisesti käytetty esitystapa on one-hot-enkoodaus. Siinä kukin token $v_j \in \mathcal{V}$ esitetään vektorina $x(v_j) \in \{0,1\}^{1\times|\mathcal{V}|}$, jonka komponentit ovat

$$x(v_j)_k = \begin{cases} 1, & \text{jos } k = j, \\ 0, & \text{muulloin.} \end{cases}$$

Kielimallinnuksessa sanastoa täydennetään usein erikoistokeneilla, joilla kuvataan sekvenssin rakennetta ja käsitellään harvinaisia havaintoja. Tällaisia ovat esimerkiksi aloitus- ja lopetusmerkit $\langle s \rangle$ ja $\langle /s \rangle$ sekä tuntemattoman tokenin merkki $\langle unk \rangle$ [17].

Aloitus- ja lopetusmerkit ovat erityisen hyödyllisiä silloin, kun mallin ennustettavaa kontekstia halutaan määritellä myös sekvenssin reuna-alueilla. Esimerkiksi n-gram-malleissa kontekstin pituus on vakio, minkä vuoksi sekvenssin alku- ja loppupäähän lisätään tyypillisesti riittävä määrä erikoistokeneita, jotta kontekstit voidaan muodostaa yhtenäisesti kaikille ajanhetkille.

Koko korpus voidaan esittää one-hot-vektoreiden jonona

$$X = (x_1, x_2, \ldots, x_{|\mathcal{W}|}),$$

missä $x_t = x(w_t) \in \{0,1\}^{1\times|\mathcal{V}|}$ on tokenin w_t one-hot-esitys. Jos syöte koostuu useasta tokenista, voidaan niiden one-hot-esitykset yhdistää esimerkiksi ketjuttamalla ne yhdeksi vektoriksi

$$\tilde{x}_t = [x_{t-n+1} \,\|\, x_{t-n+2} \,\|\, \ldots \,\|\, x_{t-1}] \in \{0,1\}^{1\times(n-1)|\mathcal{V}|},$$

missä $\|$ merkitsee vektorien vaakasuuntaista ketjuttamista. Tämä muodostaa luonnollisen syötteen monille kontekstipohjaisille malleille, sillä se säilyttää kontekstin tokenijärjestyksen eksplisiittisesti.

One-hot-enkoodaus muodostaa perustavanlaatuisen sillan tekstin ja numeeristen kielimallien välille. Se tarjoaa täsmällisen ja muodollisesti yksinkertaisen esitystavan, jonka varaan voidaan rakentaa sekä tilastollisia n-gram-malleja että myöhempiä neuroverkkopohjaisia kielimalleja.

N-gram-malli on klassinen tilastollinen kielimalli, jossa sekvenssin seuraavan tokenin todennäköisyyttä estimoidaan rajallisen kontekstin perusteella [17]. Menetelmä perustuu $(n-1)$:n kertaluvun Markov-oletukseen, jonka mukaan tokenin w_t todennäköisyys riippuu vain sitä välittömästi edeltävistä $(n-1)$ tokenista eikä koko aiemmasta historiasta. Markov-oletuksen perusteella ehdollinen todennäköisyys voidaan approksimoida muodossa

$$P(w_t \mid w_1, w_2, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \ldots, w_{t-1}),$$

missä w_t on ajanhetken t tokeni ja $(w_{t-n+1}, \ldots, w_{t-1})$ muodostaa sitä edeltävän $(n-1)$ tokenin kontekstin.

N-gram-mallien historialliset juuret yhdistetään usein Shannonin varhaiseen informaatioteoreettiseen työhön, jossa luonnollista kieltä tarkasteltiin symbolijonojen tilastollisena prosessina ja eri kertalukujen approksimaatioita havainnollistettiin englannin kielellä [36, 37].

Olkoon $\mathcal{V} = \{v_1, v_2, \ldots, v_{|\mathcal{V}|}\}$ sanasto, jossa $|\mathcal{V}|$ on sanaston koko, ja $\mathcal{W} = (w_1, w_2, \ldots, w_{|\mathcal{W}|})$ havaintoaineisto. N-gram-malli voidaan toteuttaa muodostamalla lukumatriisi $C \in \mathbb{R}^{|\mathcal{V}|^{n-1} \times |\mathcal{V}|}$, jossa $C_{c,j}$ edustaa sitä, kuinka monta kertaa kontekstia c seuraa tokeni v_j havaintoaineistossa. Formaalisti

$$C_{c,j} = \left|\{t \in \mathbb{N} \mid (w_{t-n+1}, \ldots, w_{t-1}) = c \text{ ja } w_t = v_j\}\right|.$$

Kun lukumatriisin C rivit normalisoidaan jakamalla jokainen alkio rivinsä summalla, saadaan frekvenssimatriisi $F \in \mathbb{R}^{|\mathcal{V}|^{n-1} \times |\mathcal{V}|}$, jossa

$$F_{c,j} = \frac{C_{c,j}}{\sum_{k=1}^{|\mathcal{V}|} C_{c,k}},$$

ja joka edustaa ehdollista todennäköisyyttä

$$P(w_t = v_j \mid (w_{t-n+1}, \ldots, w_{t-1}) = c).$$

Tekstin generointi n-gram-mallilla tapahtuu iteratiivisesti hyödyntämällä frekvenssimatriisia F. Ensin valitaan alkukonteksti, esimerkiksi erikoismerkeillä aloitettu $(n-1)$-tokenin pituinen alkujakso. Tämän jälkeen mallin tämänhetkistä kontekstia vastaava rivi $F_{c,\cdot}$ tulkitaan todennäköisyysjakaumaksi, josta seuraava tokeni valitaan satunnaisotannalla. Kun tokeni w_t on generoitu, konteksti päivitetään siirtämällä ikkuna yhden askeleen eteenpäin siten, että uusi konteksti muodostuu edeltävistä $(n-1)$ generoidusta tokenista. Prosessia jatketaan, kunnes generoidaan päätemerkki tai saavutetaan haluttu tekstin pituus.

Diskreetit esitykset ja frekvenssipohjaiset n-gram-mallit tarjoavat käsitteellisesti selkeän lähtökohdan kielimallinnukselle, mutta niihin liittyy kaksi perustavanlaatuista rajoitetta.

Ensinäkin one-hot-enkoodaus ilmaisee vain tokenin identiteetin. Koska erisuurten tokeneiden vektorit ovat keskenään ortogonaalisia

$$v_i \neq v_j \implies x(v_i)x(v_j)^T = 0,$$

niiden kosinisimilariteetti on poikkeuksetta nolla ja euklidinen etäisyys vakio $\sqrt{2}$. Tästä johtuen esitys ei voi mallintaa tokeneiden välisiä semanttisia tai syntaktisia suhteita [23].

Toiseksi kontekstikoon n kasvaessa lukumatriisista $C \in \mathbb{R}^{|\mathcal{V}|^{n-1} \times |\mathcal{V}|}$ tulee nopeasti erittäin suuri ja käytännöllisissä sovelluksissa hyvin harva, koska vain pieni osa kaikista mahdollisista konteksteista esiintyy tyypillisessä havaintoaineistossa. Tämä lisää sekä muistin- että laskentatehon tarvetta [17].

Tämän vuoksi on hyödyllistä tarkastella samaa ongelmaa optimointitehtävänä, mikä muodostaa luontevan siirtymän kohti jatkuvia vektoriesityksiä ja parametrisoituja neuroverkkopohjaisia malleja.

3 Jatkuvat esitykset ja neuroverkkopohjainen kielimallinnus

Siirtyminen diskreeteistä sanajonoista jatkuviin vektoriavaruuksiin ja neuroverkkopohjaiseen mallinnukseen on edistänyt merkittävästi kielimallien yleistyskykyä, sillä se mahdollistaa semanttisten suhteiden mallintamisen pelkän pintatason sanavastaavuuden sijaan [3, 23].

Tämä luo teoreettisen pohjan tutkielman keskeiselle tutkimuskysymykselle, jossa tarkastellaan mallin kykyä tulkita luonnollisen kielen tiedontarpeita luotettavasti ja johdonmukaisesti kielellisistä variaatioista huolimatta.

3.1 Upotusvektorit

One-hot-esitys tarjoaa yksikäsitteisen tavan kuvata tokeneita, mutta sen heikkoutena on, ettei se sisällä eksplisiittistä tietoa tokenien välisistä suhteista. Jokaisen tokenin vektorikuvaus on ortogonaalinen kaikkiin muihin nähden, jolloin esimerkiksi sanan *kissa* suhde sanaan *koira* on rakenteellisesti sama kuin sen suhde sanaan *tietokone*. Tämän rajoitteen ylittämiseksi käytetään upotusvektoreita (engl. *embedding vectors*), joissa kukin token kuvataan tiheällä reaaliarvoisella vektorilla.

Olkoon edelleen $\mathcal{V} = \{v_1, v_2, \ldots, v_{|\mathcal{V}|}\}$ sanasto. Tällöin tokenille v_j määritellään upotus

$$e(v_j) \in \mathbb{R}^{1\times d},$$

missä $d \ll |\mathcal{V}|$ on upotustilan dimensio.

Upotusvektorit voidaan esittää myös matriisimuodossa. Olkoon

$$E \in \mathbb{R}^{|\mathcal{V}|\times d}$$

upotusmatriisi, jonka rivit vastaavat sanaston tokeneita. Kun $x(v_j) \in \mathbb{R}^{1\times|\mathcal{V}|}$ on tokenin v_j one-hot-esitys, sen upotus saadaan muodossa

$$e(v_j) = x(v_j)E \in \mathbb{R}^{1\times d}.$$

Koska one-hot-vektori sisältää täsmälleen yhden nollasta poikkeavan komponentin, tämä kertolasku palauttaa suoraan tokenia v_j vastaavan upotusmatriisin rivin.

Upotusvektorien keskeinen etu liittyy siihen, että samankaltaisten tokenien upotukset voivat sijaita lähellä toisiaan upotustilassa. Tällöin tokenien geometrinen läheisyys voi heijastaa niiden semanttista tai syntaktista samankaltaisuutta. Esimerkiksi sanat “kuningas” ja “kuningatar” voivat saada upotukset, jotka ovat lähellä toisiaan, koska ne esiintyvät samankaltaisissa konteksteissa ja jakavat merkityksellisiä piirteitä. Upotusvektorit tarjoavat siten rikkaamman ja joustavamman esityksen tokenien välisistä suhteista kuin one-hot-esitys, jossa kaikki tokenit ovat yhtä kaukana toisistaan.

Käytännön kielimallinnuksessa upotukset opitaan tavallisesti tehtäväkohtaisesti osana laajempaa optimointia. Jos korpus on $\mathcal{W} = (w_1, w_2, \ldots, w_{|\mathcal{W}|})$, voidaan jokainen tokeni $w_t \in \mathcal{V}$ korvata sitä vastaavalla upotusvektorilla $e(w_t)$. Tällöin koko korpusta voidaan tarkastella tiheiden vektorien jonona

$$E_{\mathcal{W}} = (e(w_1), e(w_2), \ldots, e(w_{|\mathcal{W}|})).$$

Tämä esitystapa on erityisen hyödyllinen neuroverkoissa, joissa upotukset muodostavat syötteen myöhemmille kerroksille ja joissa opittavat parametrit voivat hyödyntää tokenien välisiä yhteyksiä yhteisen esitystilan kautta.

Upotusvektorit ovat myös yhteensopivia erikoistokenien kanssa. Esimerkiksi aloitusmerkki $\langle s \rangle$, lopetusmerkki $\langle /s \rangle$ ja tuntematon token $\langle unk \rangle$ voidaan sisällyttää laajennettuun sanastoon jolloin niille voidaan oppia omat upotuksensa samalla tavalla kuin muillekin tokeneille. Näin myös sekvenssien rakenteelliset elementit voidaan käsitellä yhtenäisesti muun sanaston kanssa.

Upotusvektorit muodostavat siten luonnollisen jatkeen one-hot-esitykselle. Siinä missä one-hot kuvaa tokenin pelkkänä identiteettinä, upotusvektori tarjoaa tiiviin ja optimoitavan esityksen, joka soveltuu tehokkaasti neuroverkkopohjaisiin kielimalleihin.

3.2 Upotusvektorien oppiminen Word2Vec-menetelmällä

Upotusvektorit eivät tavallisesti ole ennalta määrättyjä, vaan ne estimoidaan aineistosta optimointimenetelmän avulla. Tässä suhteessa yksi keskeisimmistä ja vaikutusvaltaisimmista menetelmistä on Word2Vec [23]. Menetelmän keskeinen tieteellinen kontribuutio ei rajoitu siihen, että sanoille opitaan tiheitä vektoriesityksiä, sillä vastaavia ajatuksia oli esitetty neuroverkkopohjaisessa kielimallinnuksessa jo aiemmin. Word2Vecin varsinainen merkitys liittyy ennen kaikkea siihen, että se osoitti tällaisten representaatioiden oppimisen olevan mahdollista laskennallisesti erittäin tehokkaalla tavalla myös hyvin suurissa korpuksissa. Lisäksi se toi esiin, että kontekstuaaliseen ennustamiseen perustuva oppiminen voi johtaa vektoriavaruuksiin, joissa ilmenee kielellisesti mielekkäitä semanttisia ja syntaktisia säännönmukaisuuksia.

Word2Vec perustuu distributionaaliseen hypoteesiin, jonka mukaan sanan merkitys voidaan päätellä sen käyttöympäristöjen perusteella. Toisin sanoen sanat, jotka esiintyvät samankaltaisissa konteksteissa, saavat oppimisprosessissa samankaltaiset vektoriesitykset. Tämä periaate on laajasti koko modernin kieliteknologian taustalla, mutta Word2Vec teki siitä käytännöllisen ja skaalautuvan laskennallisen menetelmän. Menetelmä osoitti, että diskreettien symbolien semanttista rakennetta voidaan mallintaa jatkuvassa vektoriavaruudessa pelkästään paikallisiin konteksteihin perustuvan ennustetehtävän avulla ilman käsin suunniteltuja kielellisiä piirteitä.

Olkoon korpus $\mathcal{W} = (w_1, w_2, \ldots, w_{|\mathcal{W}|})$, missä $w_t \in \mathcal{V}$ on sanaston $\mathcal{V}$ sana ajanhetkellä t. Word2Vec-malleissa tarkastellaan tyypillisesti kunkin sanan ympärille määriteltyä paikallista kontekstia, jonka laajuus määräytyy ikkunaparametrilla $c \in \mathbb{N}$. Tällöin ajanhetken t kontekstiksi voidaan määritellä

$$C_t = \{w_{t-c}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+c}\},$$

olettaen, että indeksit pysyvät korpuksen rajojen sisällä. Mikolov ym. [23] tarkastelevat kahta vaihtoehtoista perusarkkitehtuuria: jatkuvien sanojen pussimallia (*continuous bag-of-words*, CBOW) ja Skip-gram-mallia.

CBOW-mallissa pyritään ennustamaan tarkasteltava keskussana sen kontekstisanojen perusteella. Tavoitteena on siten approksimoida todennäköisyyttä

$$p(w_t \mid w_{t-c}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+c}).$$

Kontekstin sanat projisoidaan ensin vektoriesityksiksi, minkä jälkeen nämä esitykset yhdistetään tavallisesti summaamalla tai keskiarvoistamalla. Muodostettua kontekstivektoria käytetään tämän jälkeen kohdesanan ennustamiseen koko sanaston yli. CBOW-mallin etuna on sen laskennallinen tehokkuus erityisesti suurilla aineistoilla, koska useiden kontekstisanojen informaatio voidaan tiivistää yhdeksi esitykseksi ennen ennustevaihetta [23].

Skip-gram-mallissa ennustesuunta on päinvastainen. Siinä tavoitteena on ennustaa keskussanan avulla sitä ympäröivät kontekstisanat. Tällöin maksimoidaan todennäköisyydet

$$p(w_{t+j} \mid w_t), \qquad -c \leq j \leq c,\ j \neq 0.$$

Koko korpuksen tasolla vastaava tavoitefunktio voidaan kirjoittaa muodossa

$$\sum_{t=1}^{|\mathcal{W}|} \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log p(w_{t+j} \mid w_t).$$

Skip-gram-malli osoittautuu erityisen käyttökelpoiseksi silloin, kun tavoitteena on oppia informatiivisia esityksiä myös harvinaisemmille sanoille, sillä kukin havaittu sana toimii useiden kontekstisanojen ennustamisen lähtökohtana [23].

Olkoon w_I syötesana ja w_O kohdesana. Jos syötesanaa vastaava rivivektoriesitys on $v_{w_I} \in \mathbb{R}^{1\times d}$ ja kohdesanaa vastaava ulostulorivivektori on $u_{w_O} \in \mathbb{R}^{1\times d}$, voidaan todennäköisyys $p(w_O \mid w_I)$ esittää softmax-funktion avulla muodossa

$$p(w_O \mid w_I) = \frac{\exp(v_{w_I} u_{w_O}^\top)}{\sum_{w\in\mathcal{V}} \exp(v_{w_I} u_w^\top)}.$$

Tässä nimittäjän summa ulottuu koko sanastoon, mikä tekee täsmällisestä optimoinnista laskennallisesti raskasta suurilla sanastoilla. Juuri tämän ongelman ratkaiseminen on yksi Word2Vecin keskeisistä ansioista. Menetelmä teki näkyväksi sen, että suuren sanaston ulostuloprojektio muodostaa vektoriesitysten oppimisessa keskeisen laskennallisen pullonkaulan, ja esitti tähän käytännöllisiä approksimaatioita. Näistä erityisen merkittävä on negatiivinen otanta (*negative sampling*), jossa koko sanaston yli määritellyn softmax-todennäköisyyden optimoinnin sijasta ratkaistaan joukko binäärisiä luokittelutehtäviä.

Olkoon (w_I, w_O) havaittu syöte–kohdesana-pari, jossa w_O esiintyy sanan w_I kontekstissa. Negatiivisessa otannassa valitaan lisäksi K negatiivista sanaa $w_1^-, \ldots, w_K^-$ jakaumasta $P_n(w)$, joka on tyypillisesti jokin sanaston sanoille määritelty näytteenottojakauma. Tällöin mallin tavoitteena ei ole enää optimoida todennäköisyyttä $p(w_O \mid w_I)$ koko sanaston yli, vaan maksimoida havaittua paria vastaava binäärinen logistinen tavoite

$$\log \sigma\left(v_{w_I} u_{w_O}^\top\right) + \sum_{k=1}^{K} \log \sigma\left(-v_{w_I} u_{w_k^-}^\top\right),$$

missä $\sigma(x) = \frac{1}{1+e^{-x}}$ on logistinen sigmoidifunktio. Ensimmäinen termi kasvattaa havaittuun positiiviseen sana–konteksti-pariin liittyvän pistetulon $v_{w_I} u_{w_O}^\top$ arvoa, kun taas jälkimmäinen termi pienentää satunnaisesti valittujen negatiivisten sanojen

$v_{w_I} u_{w_k^-}^\top$ arvoja. Näin malli oppii erottamaan aidot kontekstisanat satunnaisista sanoista ilman, että jokaisella oppimisaskeleella tarvitsisi normalisoida jakaumaa koko sanaston yli.

Koko aineiston yli tavoite voidaan esittää odotusarvomuodossa

$$\sum_{(w_I, w_O) \in D} \left[\log \sigma\left(v_{w_I} u_{w_O}^\top\right) + \sum_{k=1}^{K} \mathbb{E}_{w_k^- \sim P_n} \log \sigma\left(-v_{w_I} u_{w_k^-}^\top\right) \right],$$

missä D on havaittujen syöte–konteksti-parien joukko ja K on negatiivisten otantojen määrä. Tällä tavoin alkuperäinen moniluokkainen softmax-optimointi approksimoidaan joukolla logistisia regressiotehtäviä, joissa erotellaan positiiviset havaintoparit negatiivisesti näytteistetyistä pareista. Laskennallinen etu syntyy siitä, että kunkin havaintoparin kohdalla päivitetään vain positiivista kohdesanaa ja K:ta negatiivista sanaa vastaavat parametrit koko sanaston kattavan päivityksen sijasta.

Word2Vec-mallissa opitaan kaksi parametrimatriisia: syötevektorit ja ulostulovektorit. Jokaiselle sanaston sanalle $v_j \in \mathcal{V}$ assosioituu siten ainakin yksi tiheä vektoriesitys. Oppimisen jälkeen sanan lopullisena esityksenä käytetään tavallisesti syötevektoria tai vaihtoehtoisesti syöte- ja ulostulovektoreiden yhdistelmää. Näin saadut upotukset heijastavat korpuksessa havaittuja yhteisesiintymisrakenteita siten, että samankaltaisissa käyttöympäristöissä esiintyvät sanat sijoittuvat tyypillisesti lähelle toisiaan upotustilassa.

Mikolov ym. [23] osoittavat lisäksi, että Word2Vecin oppimat vektoriesitykset eivät ainoastaan ryhmittele semanttisesti läheisiä sanoja, vaan niihin syntyy myös lineaarisia relaatiota vastaavia säännönmukaisuuksia. Sanavektorien erotukset voivat vastata esimerkiksi sukupuoleen, lukuun tai taivutusmuotoon liittyviä semanttisia ja syntaktisia eroja, mikä ilmenee analogiatehtävissä. Tämä havainto on menetelmän kannalta erityisen merkittävä, sillä se viittaa siihen, että kontekstuaaliseen ennustamiseen perustuva optimointi tuottaa emergentisti sellaisia vektoriavaruuden rakenteita, jotka eivät ole eksplisiittisesti koodattuja malliin, mutta jotka vastaavat kielellisesti tulkittavia suhteita. Näin Word2Vec osoitti, että jakaumallinen oppiminen ei tuota ainoastaan samankaltaisuusmittaa, vaan voi synnyttää myös abstraktimpia relaatioita mallintavia geometrisia rakenteita.

Vaikka Word2Veciä ei nykyisissä suurissa kielimalleissa tyypillisesti käytetä sellaisenaan, sen vaikutus näkyy edelleen usealla tasolla. Ensinnäkin se vakiinnutti distributionaalisen hypoteesin neuroverkkopohjaisen kielimallinnuksen keskeiseksi oppimisperiaatteeksi. Toiseksi se toi esiin suuren sanaston ulostuloprojektion laskennallisen haasteen sekä popularisoi tehokkaita approksimointimenetelmiä, kuten negatiivisen otannan. Kolmanneksi se osoitti vakuuttavasti, että yksinkertainenkin kontekstuaalinen ennustetehtävä voi johtaa vektoriavaruuksiin, joissa semanttiset ja syntaktiset suhteet ilmenevät lineaarisina rakenteina. Näistä syistä Word2Vecia voidaan pitää tärkeänä siirtymävaiheena klassisten jakaumallisten sanamallien ja myöhempien syvien neuroverkkojen välillä.

3.3 N-gram-malli optimointiongelmana

Luvussa 2 n-gram-malli esitettiin lukumääriin ja niiden normalisointiin perustuvana frekvenssimallina. Sama malli voidaan kuitenkin tulkita myös suurimman uskotta-

vuuden estimointina, mikä tekee yhteyden myöhempiin parametrisiin kielimalleihin näkyväksi.

Olkoon mallin parametreina jokaiselle kontekstille $c = (w_{t-n+1}, \ldots, w_{t-1})$ määritelty todennäköisyysjakauma

$$\theta_c = (\theta_{c,1}, \theta_{c,2}, \ldots, \theta_{c,|\mathcal{V}|}), \qquad \sum_{j=1}^{|\mathcal{V}|} \theta_{c,j} = 1, \qquad \theta_{c,j} \geq 0,$$

missä $\theta_{c,j} = P_\theta(w_t = v_j \mid c)$. Markov-oletuksen nojalla havaintoaineiston $\mathcal{W} = (w_1, \ldots, w_{|\mathcal{W}|})$ uskottavuus voidaan kirjoittaa muodossa

$$P(\mathcal{W}; \theta) \approx \prod_{t=1}^{|\mathcal{W}|} P_\theta(w_t \mid w_{t-n+1}, \ldots, w_{t-1}).$$

Kun aineisto ryhmitellään kontekstin c ja sitä seuraavan tokenin v_j mukaan, saadaan

$$P(\mathcal{W}; \theta) = \prod_c \prod_{j=1}^{|\mathcal{V}|} \theta_{c,j}^{C_{c,j}},$$

missä $C_{c,j}$ on niiden havaintojen lukumäärä, joissa kontekstia c seuraa tokeni v_j.

Koneoppimisessa optimointi esitetään tavallisesti minimointitehtävänä, joten määritellään häviöksi negatiivinen log-uskottavuus

$$J(\theta; \mathcal{W}) = -\log P(\mathcal{W}; \theta) = -\sum_c \sum_{j=1}^{|\mathcal{V}|} C_{c,j} \log \theta_{c,j}.$$

Koska parametrit θ_c esiintyvät vain omaa kontekstiaan vastaavissa termeissä, optimointiongelma hajoaa erillisiksi kontekstikohtaisiksi osatehtäviksi:

$$\min_{\theta_c} -\sum_{j=1}^{|\mathcal{V}|} C_{c,j} \log \theta_{c,j} \qquad \text{s.e.} \qquad \sum_{j=1}^{|\mathcal{V}|} \theta_{c,j} = 1.$$

Tämä voidaan ratkaista Lagrangen menetelmällä. Määritellään

$$\mathcal{L}_{\text{Lag}}(\theta_c, \lambda) = \sum_{j=1}^{|\mathcal{V}|} C_{c,j} \log \theta_{c,j} + \lambda \left(1 - \sum_{j=1}^{|\mathcal{V}|} \theta_{c,j}\right).$$

Derivoimalla $\theta_{c,j}$:n suhteen saadaan

$$\frac{\partial \mathcal{L}_{\text{Lag}}}{\partial \theta_{c,j}} = \frac{C_{c,j}}{\theta_{c,j}} - \lambda = 0,$$

mistä seuraa

$$\theta_{c,j} = \frac{C_{c,j}}{\lambda}.$$

Kun tämä sijoitetaan normalisaatioehtoon

$$\sum_{j=1}^{|\mathcal{V}|} \theta_{c,j} = 1,$$

saadaan

$$\lambda = \sum_{k=1}^{|\mathcal{V}|} C_{c,k},$$

ja siten

$$\theta_{c,j} = \frac{C_{c,j}}{\sum_{k=1}^{|\mathcal{V}|} C_{c,k}}.$$

Tulos osoittaa, että suurimman uskottavuuden estimaatti on täsmälleen sama kuin luvussa 2 määritelty rivikohtaisesti normalisoitu frekvenssimatriisi F. Toisin sanoen klassinen frekvenssipohjainen n-gram-malli voidaan tulkita parametrisen mallin maksimiuskottavuusratkaisuna.

Tämä näkökulma tekee samalla näkyväksi mallin keskeisen rajoitteen, sillä jokaiselle mahdolliselle kontekstille tarvitaan oma erillinen todennäköisyysjakaumansa. Kontekstin pituuden kasvaessa parametrien määrä kasvaa nopeasti, jolloin malli muuttuu harvaksi ja yleistää heikosti havaitsemattomiin konteksteihin. Kielellisen variaation ja lauserakenteiden monimuotoisuuden käsittely on tällaiselle taulukoidulle mallille haastavaa, sillä toisiaan merkitykseltään lähellä olevat ilmaisut eivät jaa keskenään informaatiota. Tästä syystä on perusteltua siirtyä malleihin, joissa ehdolliset todennäköisyydet opitaan jaetun parametrisaation avulla jatkuvassa avaruudessa.

3.4 Monikerroksinen perseptroni seuraavan tokenin ennustajana

Seuraavan tokenin ehdollista jakaumaa voidaan approksimoida parametrisoidulla neuroverkolla. Yksi yksinkertainen ja historiallisesti keskeinen ratkaisu tähän on monikerroksinen perseptroni (engl. *multilayer perceptron*, MLP).

MLP on eteenpäinsyöttävä keinotekoinen neuroverkko, joka koostuu syötekerroksesta, yhdestä tai useammasta piilokerroksesta sekä ulostulokerroksesta. Sen historialliset juuret ovat perseptronitutkimuksessa [32, 33]. Näissä töissä esitettiin varhaisia perseptronimalleja ja kerroksellisia rakenteita, mutta tehokas yleiskäyttöinen menetelmä kaikkien kerrosten parametrien oppimiseen puuttui vielä.

Neuroverkkojen varhaista kehitystä hidasti merkittävästi kirja, jossa analysoitiin yksikerroksisten perseptronien rajoitteita[25]. Teos osoitti, että yksinkertainen perseptroni ei voi ratkaista tiettyjä lineaarisesti erottamattomia ongelmia, kuten XOR-tehtävää. Vaikka kritiikki kohdistui ensisijaisesti yksikerroksisiin malleihin, sen vaikutus ulottui laajemmin neuroverkkotutkimukseen ja osaltaan vähensi kiinnostusta monikerroksisiin verkkoihin useiden vuosien ajaksi.

Monikerroksisten verkkojen tehokkaan opettamisen kannalta keskeinen teoreettinen lähtökohta syntyi työssä, jossa esitettiin käänteisen suunnan automaattinen derivointi menetelmä[20]. Tätä ajatusta sovellettiin myöhemmin neuroverkkoihin, jossa termi vastavirta-algoritmi (engl. *backpropagation*) vakiintui [45, 34].

MLP voidaan ymmärtää funktion approksimaattorina, joka muodostaa kuvauksen syöteavaruudesta tulosteavaruuteen. Tämä kuvaus toteutetaan yhdistettynä funktiona, joka koostuu peräkkäisistä affiineista muunnoksista ja niitä seuraavista epälineaarisista aktivaatiofunktioista. Tämän ansiosta MLP kykenee mallintamaan sellaisia syötteen ja tulosteen välisiä riippuvuuksia, joita lineaarinen malli ei voi esittää.

Mallin toimintaa ohjaavat opittavat parametrit, tyypillisesti painomatriisit ja biasvektorit, joiden arvot estimoidaan datasta minimoimalla valittua kohdefunktiota numeerisilla optimointimenetelmillä, kuten gradienttimenettelyllä [13].

Olkoon $c = (w_{t-n+1}, \ldots, w_{t-1})$ tarkasteltava konteksti. Monikerroksisen perseptronin syötevektori $x(c) \in \mathbb{R}^{1 \times d_0}$, missä $d_0 = (n-1)d$, muodostetaan ketjuttamalla kontekstin tokenien upotusvektorit:

$$x(c) = [e(w_{t-n+1}) \,\|\, e(w_{t-n+2}) \,\|\, \ldots \,\|\, e(w_{t-1})],$$

missä $\|$ merkitsee vektorien vaakasuuntaista ketjuttamista. Monikerroksinen perseptroni muuntaa tämän syötteen vaiheittain piirre-esitykseksi soveltamalla peräkkäisiä affiineja muunnoksia ja epälineaarisia aktivaatioita. Jos mallissa on L piilokerrosta, voidaan sen laskenta esittää muodossa

$$\begin{aligned} h^{(0)} &= x(c), \\ a^{(i)} &= h^{(i-1)} W^{(i)} + b^{(i)}, && i = 1, \ldots, L, \\ h^{(i)} &= \phi^{(i)}\left(a^{(i)}\right), && i = 1, \ldots, L, \end{aligned}$$

missä $W^{(i)} \in \mathbb{R}^{d_{i-1} \times d_i}$ on kerroksen i painomatriisi, $b^{(i)} \in \mathbb{R}^{1 \times d_i}$ sen bias-rivivektori, $a^{(i)} \in \mathbb{R}^{1 \times d_i}$ affiinisen muunnoksen tulos ja $h^{(i)} \in \mathbb{R}^{1 \times d_i}$ kerroksen i piiloesitys rivivektorina. Funktio $\phi^{(i)}$ on kerroksen epälineaarinen aktivaatiofunktio.

Viimeisestä piilokerroksesta muodostetaan ulostulon normalisoimattomat pistemäärät eli logitit rivivektorina

$$\ell = h^{(L)} W^{(\text{out})} + b^{(\text{out})},$$

missä $W^{(\text{out})} \in \mathbb{R}^{d_L \times |\mathcal{V}|}$, $b^{(\text{out})} \in \mathbb{R}^{1 \times |\mathcal{V}|}$ ja $\ell \in \mathbb{R}^{1 \times |\mathcal{V}|}$. Vektorin ℓ komponentti ℓ_j vastaa sanaston tokenin v_j normalisoimatonta pistemäärää. Näistä muodostetaan ehdollinen todennäköisyysjakauma softmax-funktiolla:

$$p_\theta(w_t = v_j \mid c) = \frac{\exp(\ell_j)}{\sum_{k=1}^{|\mathcal{V}|} \exp(\ell_k)}, \qquad j = 1, \ldots, |\mathcal{V}|.$$

MLP muuntaa kontekstin ensin yhden tai usean kerroksen kautta abstraktiksi piirre-esitykseksi, jonka perusteella se laskee jokaiselle sanaston tokenille pistemäärän ja normalisoi nämä pistemäärät todennäköisyysjakaumaksi.

Mallin oppiminen perustuu samaan negatiiviseen log-uskottavuuden minimointiin kuin n-gram-mallissa:

$$J(\theta) = -\frac{1}{|\mathcal{W}|} \sum_{t=1}^{|\mathcal{W}|} \log p_\theta(w_t \mid w_{t-n+1}, \ldots, w_{t-1}).$$

Erona perinteiseen frekvenssipohjaiseen n-gram-malliin, joka estimoi jokaiselle mahdolliselle diskreetille kontekstille oman erillisen parametrivektorinsa, MLP approksimoi todennäköisyyksiä jatkuvassa avaruudessa. Koska erilliset tokenit projisoidaan matalaulotteisiksi tiheiksi upotusvektoreiksi $v \in \mathbb{R}^{1 \times d}$, malli oppii esittämään sanojen semanttisia ja syntaktisia samankaltaisuuksia vektoriesitysten avulla.

MLP:n approksimoima funktio on jatkuva ja tyypillisesti sileä siinä mielessä, että pienet muutokset syöteavaruudessa voivat johtaa samankaltaisiin ulostulojakaumiin. Tästä syystä malli kykenee yleistämään myös harvinaisiin tai aiemmin havaitsemattomiin konteksteihin hyödyntämällä sellaisten koulutusdatassa esiintyneiden kontekstien rakennetta, joiden esitykset sijaitsevat lähellä niitä yhteisessä avaruudessa [3].Tämä ominaisuus on keskeinen käyttäjän tiedontarpeiden tulkinnassa, sillä se antaa mallille valmiuden yleistää kielellisiin variaatioihin ja synonyymisiin ilmaisuihin ilman erillisiä tasoitusmenetelmiä [16].

Opittavat painomatriisit ja bias-vektorit jakautuvat kaikkien kontekstien kesken, joten jokainen harjoitusnäyte päivittää mallin kokonaisesitystä. Parametrien jakaminen tekee neuroverkkopohjaisesta lähestymistavasta joustavamman ja muistikapasiteetiltaan tehokkaamman kuin diskreetti taulukkoesitys [3, 13, 17]. Samalla se luo arkkitehtuurisen perustan laajemmille malleille, joissa syötteen vaihtelevuudesta huolimatta pyritään säilyttämään tuotetun tulosteen semanttinen eheys.

4 Gradienttimenetelmä

Edellisessä luvussa kielimallinnus muotoiltiin optimointiongelmaksi, jossa mallin parametrit valitaan siten, että ennustettu seuraavan tokenin jakauma vastaa mahdollisimman hyvin opetusaineistoa. Tässä luvussa tarkastellaan, millä menetelmillä tällainen ongelma ratkaistaan käytännössä. Tarkastelu on tutkielman kannalta olennainen, sillä kielimallin tuottama SQL-kysely on viime kädessä näyte jakaumasta, jonka muodon optimointi on määrännyt.

Optimointi ei tuota eksplisiittistä sääntöjärjestelmää kyselyiden muodostamiseen, vaan parametrit, jotka minimoivat keskimääräistä ennustevirhettä suuressa aineistossa. Tämä selittää osaltaan, miksi järjestelmän tulkinta voi vaihdella merkitykseltään samanlaisten kysymysten välillä.

Gradienttimenetelmä ja sen stokastinen muunnelma ovat ensimmäisen kertaluvun optimointimenetelmiä, joissa kohdefunktion gradienttia hyödynnetään parametrivektorin päivittämisessä. Tarkastellaan minimointitehtävää

$$\min_{\theta \in \mathbb{R}^d} J(\theta),$$

missä $J : \mathbb{R}^d \to \mathbb{R}$ on jatkuvasti derivoituva kohdefunktio ja $\theta \in \mathbb{R}^d$ sen parametrivektori. Gradienttimenetelmän perusajatus perustuu kohdefunktion ensimmäisen kertaluvun paikalliseen approksimaatioon. Olkoon $\theta_t \in \mathbb{R}^d$ ajanhetken t parametrivektori ja $v \in \mathbb{R}^d$ pieni siirtymävektori. Taylorin lauseen nojalla saadaan

$$J(\theta_t + v) = J(\theta_t) + \langle \nabla J(\theta_t), v \rangle + o(\|v\|_2), \qquad \text{kun } \|v\|_2 \to 0.$$

Tästä seuraa, että funktion arvon paikallista muutosta approksimoi ensisijaisesti lineaarinen termi $\langle \nabla J(\theta_t), v \rangle$. Rajoittamalla siirtymän pituutta ehdolla $\|v\|_2 \leq \varepsilon$ saadaan minimointitehtävä

$$\min_{\|v\|_2 \leq \varepsilon} \langle \nabla J(\theta_t), v \rangle.$$

Olettamalla, että $\nabla J(\theta_t) \neq 0$, Cauchy–Schwarzin epäyhtälöstä seuraa arvio

$$\langle \nabla J(\theta_t), v \rangle \geq -\|\nabla J(\theta_t)\|_2 \|v\|_2 \geq -\varepsilon \|\nabla J(\theta_t)\|_2.$$

Yhtäsuuruus toteutuu, kun vektori v on vastakkaissuuntainen gradienttiin nähden. Näin ollen lineaarisen approksimaation kannalta optimaalinen siirtymä on

$$v^* = -\frac{\varepsilon}{\|\nabla J(\theta_t)\|_2} \nabla J(\theta_t).$$

Optimaalinen siirtymä v^* osoittaa, että funktion arvo pienenee paikallisesti nopeimmin negatiivisen gradientin suuntaan. Uusi parametriarvo saadaan asettamalla $\theta_{t+1} = \theta_t + v^*$. Käytännössä normalisointitermi $\varepsilon / \|\nabla J(\theta_t)\|_2$ korvataan usein suoraan oppimisnopeudella $\eta > 0$ (engl. *learning rate*), jolloin askelpituus skaalautuu luonnostaan gradientin suuruuden mukaan. Tämä johtaa gradienttimenetelmän iteratiiviseen päivityssääntöön

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t).$$

Koneoppimisessa kohdefunktio muodostetaan tyypillisesti empiirisenä riskinä

$$J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(\theta; z_i),$$

missä $z_1, \ldots, z_N$ ovat opetusdatan näytteitä ja $\ell(\theta; z_i)$ on havaintoon z_i liittyvä häviöfunktio. Tällöin täysi gradientti on

$$\nabla J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla \ell(\theta; z_i).$$

Kielimallien opetuksessa käytettävät aineistot ovat usein erittäin suuria, minkä vuoksi täyden gradientin laskeminen jokaisella iteraatiolla on laskennallisesti kallista. Tämän vuoksi käytetään stokastista gradienttimenetelmää (engl. *stochastic gradient descent*, SGD), jossa täysi gradientti korvataan satunnaisesti poimitun mini-erän perusteella muodostetulla gradienttiestimaatilla [31, 4, 5].

Olkoon $B_t \subset \{1, \ldots, N\}$ hetkellä t muodostettu satunnainen mini-erä, jonka koko on $|B_t| = b$. Tällöin gradienttiestimaatti g_t määritellään muodossa

$$g_t = \frac{1}{b} \sum_{i \in B_t} \nabla \ell(\theta_t; z_i),$$

ja SGD:n päivityssääntö on

$$\theta_{t+1} = \theta_t - \eta g_t.$$

Jos mini-erä valitaan hetkellä t tasaisesti kaikkien b-alkioisten indeksijoukkojen joukosta, gradienttiestimaatti on ehdollisen odotusarvon mielessä harhaton [5]

$$\mathbb{E}[g_t \mid \theta_t] = \nabla J(\theta_t).$$

Stokastinen gradientti vastaa siten keskimäärin täyttä gradienttia, vaikka yksittäinen päivitys sisältää satunnaisuudesta johtuvaa kohinaa. Tämä kohina estää yleensä kohdefunktion arvon monotonisen pienenemisen, mutta mini-erien käyttö lieventää laskennallisia haasteita [4, 13].

Sekä gradienttimenetelmän että SGD:n päivitykset perustuvat kohdefunktion ensimmäisen kertaluvun paikalliseen approksimaatioon. Tämä approksimaatio ei kuitenkaan sellaisenaan takaa kohdefunktion arvon pienenemistä. Tällaisen takuun johtamiseksi tarvitaan gradientin säännöllisyyttä koskevia lisäoletuksia.

Määritelmä 1. Jatkuvasti derivoituva funktio $J : \mathbb{R}^d \to \mathbb{R}$ on L-sileä, jos sen gradientti ∇J on L-Lipschitz, eli kaikilla $\theta, \theta' \in \mathbb{R}^d$ pätee

$$\|\nabla J(\theta) - \nabla J(\theta')\|_2 \leq L \|\theta - \theta'\|_2.$$

Olettaen, että kohdefunktio on L-sileä, sen muutokselle voidaan johtaa neliöllinen yläraja.

Lemma 1 (Neliöllinen yläraja)**.** *Olkoon $J : \mathbb{R}^d \to \mathbb{R}$ L-sileä funktio. Tällöin kaikilla $\theta, h \in \mathbb{R}^d$ pätee*

$$J(\theta + h) \leq J(\theta) + \langle \nabla J(\theta), h \rangle + \frac{L}{2} \|h\|_2^2.$$

Todistus. Analyysin peruslauseesta seuraa, että

$$J(\theta + h) - J(\theta) = \int_0^1 \langle \nabla J(\theta + th), h \rangle \, dt.$$

Lisäämällä ja vähentämällä termi $\langle \nabla J(\theta), h \rangle$ integraalin sisällä saadaan

$$\begin{aligned} J(\theta + h) - J(\theta) &= \int_0^1 \langle \nabla J(\theta), h \rangle \, dt \\ &\quad + \int_0^1 \langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle \, dt \\ &= \langle \nabla J(\theta), h \rangle + \int_0^1 \langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle \, dt. \end{aligned}$$

Cauchy–Schwarzin epäyhtälön ja L-Lipschitz-oletuksen perusteella integraalissa oleva termi voidaan arvioida ylhäältä

$$\begin{aligned} \langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle &\leq \|\nabla J(\theta + th) - \nabla J(\theta)\|_2 \|h\|_2 \\ &\leq Lt\|h\|_2^2. \end{aligned}$$

Integroimalla saadaan

$$\begin{aligned} \int_0^1 \langle \nabla J(\theta + th) - \nabla J(\theta), h \rangle \, dt &\leq \int_0^1 Lt\|h\|_2^2 \, dt \\ &= \frac{L}{2}\|h\|_2^2. \end{aligned}$$

Näin ollen

$$J(\theta + h) \leq J(\theta) + \langle \nabla J(\theta), h \rangle + \frac{L}{2}\|h\|_2^2.$$

□

Neliöllinen yläraja tarjoaa keskeisen työkalun optimointimenetelmien analysointiin. Soveltamalla sitä gradienttimenetelmän päivitysaskeleeseen, jossa parametreja siirretään negatiivisen gradientin suuntaan askelpituudella $\eta > 0$, saadaan taattua kohdefunktion arvon pieneneminen kullakin iteraatiolla. Tämä tulos tunnetaan laskeutumislemmanna.

Lemma 2 (Laskeutumislemma)**.** *Olkoon $J : \mathbb{R}^d \to \mathbb{R}$ L-sileä funktio ja $0 < \eta \leq 1/L$. Tällöin gradienttimenetelmän päivityksellä*

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

pätee

$$J(\theta_{t+1}) \leq J(\theta_t) - \frac{\eta}{2}\|\nabla J(\theta_t)\|_2^2.$$

Todistus. Sovelletaan neliöllistä ylärajaa (lemma 1) sijoittamalla $\theta = \theta_t$ ja $h = -\eta \nabla J(\theta_t)$, jolloin $\theta + h = \theta_{t+1}$. Tällöin saadaan

$$\begin{aligned} J(\theta_{t+1}) &\leq J(\theta_t) + \langle \nabla J(\theta_t), -\eta \nabla J(\theta_t) \rangle + \frac{L}{2} \| - \eta \nabla J(\theta_t)\|_2^2 \\ &= J(\theta_t) - \eta \|\nabla J(\theta_t)\|_2^2 + \frac{L\eta^2}{2} \|\nabla J(\theta_t)\|_2^2 \\ &= J(\theta_t) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla J(\theta_t)\|_2^2. \end{aligned}$$

Koska $0 < \eta \leq 1/L$, pätee $1 - \frac{L\eta}{2} \geq \frac{1}{2}$, mistä seuraa

$$J(\theta_{t+1}) \leq J(\theta_t) - \frac{\eta}{2} \|\nabla J(\theta_t)\|_2^2.$$

□

Lemma osoittaa, että kohdefunktion arvo pienenee jokaisessa ei-stationaarisessa pisteessä, kun oppimisnopeus valitaan ehdon $0 < \eta \leq 1/L$ mukaisesti. Käytännössä oppimisnopeuden valinta ei kuitenkaan ole suoraviivaista, sillä kohdefunktion kaarevuus voi vaihdella merkittävästi eri parametrisuunnissa. Seuraavassa alaluvussa tarkastellaan tämän vaihtelun aiheuttamia haasteita optimoinnille.

4.1 Häiriöalttius ja toisen kertaluvun geometria

Gradienttipohjaisten menetelmien keskeinen haaste on, että kohdefunktion lokaali kaarevuus voi vaihdella huomattavasti eri parametrisuunnissa. Tätä ilmiötä kutsutaan numeeriseksi häiriöalttiudeksi (engl. *ill-conditioning*). Olkoon kohdefunktio J L-sileä ja kahdesti jatkuvasti derivoituva. Tällöin pisteen θ_t ympäristössä pätee toisen kertaluvun Taylorin approksimaatio

$$J(\theta_t + \Delta\theta) \approx J(\theta_t) + \langle \nabla J(\theta_t), \Delta\theta \rangle + \frac{1}{2} \langle \Delta\theta, \nabla^2 J(\theta_t) \Delta\theta \rangle.$$

Merkitään Hessin matriisia pisteessä θ_t symbolilla

$$H_t = \nabla^2 J(\theta_t).$$

Koska H_t on symmetrinen, sen ominaisarvot ovat reaalisia. Oletetaan tässä, että lokaali kvadraattinen malli on konveksi eli $H_t \succeq 0$. Tällöin ominaisarvot voidaan järjestää muodossa

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0,$$

missä λ_1 vastaa suurinta ja λ_d pienintä paikallista kaarevuutta.

Funktion L-sileyden nojalla Hessin operaattorinormille pätee $\|H_t\|_2 \leq L$, jolloin $\lambda_1 \leq L$. Tämä rajoittaa oppimisnopeuden suuruutta, koska liian suuri askel voi johtaa divergenssiin tai voimakkaaseen oskillaatioon suurimman kaarevuuden suunnassa. Häiriöalttiuden vaikutusta suppenemiseen voidaan havainnollistaa tarkastelemalla stationaarisen pisteen θ^* ympäristöä, jossa $\nabla J(\theta^*) = 0$. Merkitsemällä

$H^* = \nabla^2 J(\theta^*)$ ja virhevektoria $e_t = \theta_t - \theta^*$ saadaan lokaaliksi linearisoiduksi virhedynamiikaksi

$$e_{t+1} \approx (I - \eta H^*) e_t.$$

Kun virhevektori esitetään H^*:n ortonormaalissa ominaisvektorikannassa, kukin virhekomponentti supistuu kertoimella

$$e_{t+1}^{(k)} \approx (1 - \eta\lambda_k) e_t^{(k)}, \qquad k = 1, \ldots, d.$$

Yhtä oppimisnopeutta käytettäessä syntyy siten perustavanlaatuinen kompromissi. Jos η valitaan suureksi ($\eta \approx 1/\lambda_1$) nopean etenemisen saavuttamiseksi jyrkimmän kaarevuuden suunnassa, loivimmissa suunnissa pätee $1 - \eta\lambda_d \approx 1$. Tällöin virhe näissä suunnissa supistuu hyvin hitaasti. Ongelman vakavuutta kuvaa Hessin matriisin ehtoluku

$$\kappa = \frac{\lambda_1}{\lambda_d},$$

kun $\lambda_d > 0$. Suuri ehtoluku tarkoittaa, että gradienttimenetelmä etenee kapeassa laaksossa oskilloiden voimakkaan kaarevuuden suunnassa ja vain hitaasti laakson pohjan suuntaisesti [26].

Toisen kertaluvun optimointimenetelmät, kuten Newtonin menetelmä, hyödyntävät paikallista kaarevuusinformaatiota skaalaamalla päivitysaskelen käänteisellä Hessin matriisilla

$$\theta_{t+1} = \theta_t - \eta H_t^{-1} \nabla J(\theta_t).$$

Tällöin päivityssuunta mukautuu sekä eri suuntien kaarevuuseroihin että parametrien välisiin riippuvuuksiin, joita Hessin matriisin ei-diagonaaliset alkiot kuvaavat. Nykyaikaisissa neuroverkoissa ja kielimalleissa parametrien lukumäärä d on kuitenkin tyypillisesti miljoonia tai miljardeja. Täyden Hessin matriisin tallentaminen vaatii $\mathcal{O}(d^2)$ muistia, ja sen kääntäminen vaatii $\mathcal{O}(d^3)$ laskutoimitusta. Tästä syystä täyteen kaarevuusinformaatioon perustuvien menetelmien suora soveltaminen on suuressa mittakaavassa yleensä epäkäytännöllistä.

4.2 Adaptiiviset optimointimenetelmät

Syväoppimisessa hyödynnetään tämän vuoksi laajasti ensimmäisen kertaluvun optimointimenetelmiä, jotka pyrkivät parantamaan gradienttimenetelmien käytännön toimintaa pienellä laskennallisella lisäkustannuksella [13]. Keskeisiä menetelmäperheitä ovat *momentum*-menetelmät, jotka vaimentavat häiriöalttiuden aiheuttamaa oskillointia suodattamalla päivityssuuntaa ajassa, sekä adaptiiviset optimointimenetelmät [18].

Adaptiivisten menetelmien keskeinen periaate on muodostaa parametrikohtaisia oppimisnopeuksia gradienttihistorian perusteella. Menetelmät mukauttavat päivitysten mittakaavaa erityisesti silloin, kun gradienttien suuruus vaihtelee huomattavasti parametrikoordinaattien välillä. Syvissä neuroverkoissa eri kerrosten parametrit sekä esimerkiksi harvoin esiintyviä sanoja vastaavat upotusvektorit voivat

saada hyvin erisuuruisia gradientteja [12]. Yhden globaalin oppimisnopeuden käyttö voi tällöin johtaa siihen, että suuria gradientteja saavat parametrit päivittyvät liian voimakkaasti, kun taas pieniä gradientteja saavat parametrit etenevät hitaasti.

Adaptiiviset menetelmät tasapainottavat tätä vaihtelua skaalaamalla kunkin parametrin päivitystä sen aiemmin saamien gradienttien perusteella. Tämä voidaan tulkita diagonaalisena esiehdollistuksena. Jokaiselle parametrille määritetään oma päivitysaskelen mittakaava ilman, että parametrien välisiä riippuvuuksia tarvitsee käsitellä eksplisiittisesti [5, 12]. Tämä tekee menetelmistä laskennallisesti tehokkaita ja erityisen soveltuvia suuriin, korkeaulotteisiin optimointiongelmiin.

Yleinen adaptiivinen menetelmä voidaan esittää muodossa

$$\theta_{t+1} = \theta_t - \eta_t D_t g_t,$$

missä g_t on hetkellä t laskettu stokastinen gradientti, $\eta_t > 0$ on globaali oppimisnopeus ja $D_t \in \mathbb{R}^{d\times d}$ on positiivisesti definiitti diagonaalinen skaalausmatriisi, joka muodostetaan aiempien gradienttien perusteella. Diagonaalisuuden vuoksi menetelmien muisti- ja aikavaativuus on vain $\mathcal{O}(d)$. Tämä mahdollistaa akselinsuuntaisten mittakaavaerojen tehokkaan tasaamisen, vaikka se ei korjaakaan koordinaatiston rotaatioista johtuvaa yleistä häiriöalttiutta.

Adaptiivisten menetelmien kehitys syväoppimisessa on rakentunut erityisesti AdaGradin, RMSPropin ja Adamin ympärille. AdaGrad tuo esiin parametrikohtaisen oppimisnopeuden perusidean, RMSProp korjaa AdaGradin käytännöllistä heikkoutta ja Adam yhdistää adaptiivisen skaalaamisen sekä momentumin. Modernissa neuroverkkojen ja erityisesti transformer-arkkitehtuurien opetuksessa myös AdamW on saavuttanut vakiintuneen aseman [12, 42, 18, 22, 43].

Momentum. Ennen puhtaasti adaptiivisia menetelmiä stokastisen gradienttimenetelmän keskeisimmäksi laajennukseksi vakiintui momentum- eli liikemäärämenetelmä [41]. Menetelmän tarkoituksena on nopeuttaa optimointia loivilla alueilla sekä vaimentaa oskillaatiota suuntiin, joissa kohdefunktion kaarevuus on suurta. Tämä saavutetaan lisäämällä päivitykseen liukuma aiemmista siirtymistä

$$v_t = \gamma v_{t-1} + \eta g_t,$$

missä $\gamma \in [0, 1)$ on momentum-kerroin. Parametripäivitys tehdään kertyneen nopeusvektorin v_t suunnassa

$$\theta_{t+1} = \theta_t - v_t.$$

Vaihtoehtoisesti momentum voidaan formuloida eksponentiaalisesti painotettuna gradienttien liukuvana keskiarvona $m_t = \beta m_{t-1} + (1 - \beta)g_t$, jolloin päivitys on muotoa $\theta_{t+1} = \theta_t - \eta m_t$. Tämä jälkimmäinen esitysmuoto toimii suoraan perustana Adamin ensimmäisen momentin laskennalle.

AdaGrad. AdaGrad (*adaptive gradient*) oli ensimmäisiä laajasti tunnettuja menetelmiä, joissa oppimisnopeus mukautetaan parametrikohtaisesti gradienttihistorian

perusteella [12]. Menetelmässä ylläpidetään neliöityjen gradienttien kumulatiivista summaa

$$s_t = \sum_{\tau=1}^{t} g_\tau^2,$$

missä potenssi- ja neliöjuurioperaatiot sekä jakolasku tulkitaan komponenttikohtaisesti. Päivitys kirjoitetaan muodossa

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \varepsilon},$$

missä $\varepsilon > 0$ on pieni vakio numeerisen stabiiliuden varmistamiseksi.

AdaGradin keskeinen ominaisuus on, että parametreille, joiden gradientit ovat toistuvasti suuria, kertyy nopeasti suuri nimittäjä, jolloin niiden efektiivinen oppimisnopeus pienenee. Vastaavasti parametrit, joiden gradientit ovat harvinaisia tai pieniä, voivat saada suhteellisesti suurempia päivityksiä. Tämän vuoksi menetelmä soveltuu erityisesti tilanteisiin, joissa gradientit ovat harvoja. AdaGradin keskeinen heikkous on kuitenkin se, että kertymä s_t kasvaa monotonisesti. Tämän seurauksena askelpituudet voivat ajan myötä pienentyä liikaa ja optimointi hidastua olennaisesti.

RMSProp. RMSProp (*root mean square propagation*) kehitettiin käytännölliseksi korjaukseksi AdaGradin liian nopeasti pieneneville askelpituuksille. Menetelmää ei alun perin esitelty vertaisarvioidussa tieteellisessä artikkelissa, vaan se tuotiin julkisuuteen Geoffrey Hintonin pitämällä Coursera-verkkokurssilla [42]. RMSPropin toimintaperiaate on sittemmin dokumentoitu laajasti alan perusteoksissa [13].

Sen sijaan, että neliöityjä gradientteja summattaisiin koko optimoinnin ajalta, RMSProp käyttää eksponentiaalisesti painotettua liukuvaa keskiarvoa

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2,$$

missä $\rho \in (0, 1)$ on muistiparametri. Päivityssääntö on

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{v_t} + \varepsilon}.$$

RMSProp säilyttää AdaGradin perusintuition parametrikohtaisesta skaalaamisesta, mutta käyttää pitkän aikavälin kertymän sijasta paikallisempaa estimaattia gradienttien mittakaavasta. Tämä estää oppimisnopeuden systemaattisen pienentymisen ja tekee menetelmästä käytännössä käyttökelpoisemman syvien neuroverkkojen opetuksessa [13].

Adam. Adam (*adaptive moment estimation*) yhdistää gradientin ensimmäisen momentin eli suunnatun liukuvan keskiarvon sekä neliöityjen gradienttien toisen momentin eli liukuvan keskiarvon [18]. Menetelmässä määritellään rekursiot

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

missä $\beta_1, \beta_2 \in (0, 1)$ ovat muistiparametreja. Koska m_t ja v_t alustetaan tavallisesti nollaksi, ne ovat alkuvaiheessa harhaisia kohti nollaa. Tämän vuoksi käytetään harhakorjattuja estimaatteja

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t},$$
$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Päivitys saa tällöin muodon

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Adamia voidaan tulkita menetelmäksi, jossa m_t vakauttaa päivityssuuntaa momentum-tyyppisesti, kun taas v_t skaalaa askelpituutta parametrikohtaisesti gradienttien havaittuun mittakaavaan nähden. Tämä tekee menetelmästä käytännössä usein tehokkaan ja suhteellisen robustin oppimisnopeuden valinnalle. Näistä syistä Adamista on tullut yksi käytetyimmistä optimointimenetelmistä syväoppimisessa.

Adam ei kuitenkaan ole ongelmaton. On havaittu, että adaptiiviset menetelmät eivät kaikissa tilanteissa tuota yhtä hyvää yleistymistä kuin huolellisesti viritetty SGD momentumilla [46]. Lisäksi Adamin alkuperäinen algoritmi voi tietyillä gradienttisekvensseillä epäonnistua konvergenssissa, minkä vuoksi sen konvergenssianalyysiä on myöhemmin tarkennettu esimerkiksi AMSGrad-menetelmän avulla [30].

Adaptiivisuutta ei siten tule tulkita yleispätevänä ratkaisuna kaikkiin optimoinnin haasteisiin, vaan käytännöllisenä keinona muokata päivityksen geometriaa ja vähentää oppimisnopeuden herkkyyttä.

AdamW. AdamW on Adamin muunnelma, jossa painojen kutistaminen (*weight decay*) erotetaan adaptiivisesta gradienttipäivityksestä [22]. Tavallisessa L_2-regularisoinnin ja Adamin yhdistelmässä regularisaatiotermi sekoittuu adaptiiviseen skaalaukseen tavalla, joka ei vastaa puhdasta painojen kutistamista. AdamW:ssä parametripäivitys kirjoitetaan tyypillisesti muodossa

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} - \eta \lambda \theta_t,$$

missä $\lambda \geq 0$ säätelee painojen kutistamisen voimakkuutta.

Tämä erotus on osoittautunut käytännössä tärkeäksi erityisesti syvien neuroverkkojen opetuksessa [22]. AdamW:tä pidetäänkin usein tarkoituksenmukaisempana valintana kuin alkuperäistä Adamia silloin, kun mallin opetuksessa käytetään eksplisiittistä painojen kutistamista. Transformer-arkkitehtuurien opetuksessa Adam-tyyppiset adaptiiviset menetelmät ovat yleisesti käytettyjä [43].

Lopuksi on syytä huomata, että hyvin suurissa kielimalleissa myös optimointimenetelmän muistikustannus muodostuu olennaiseksi. Adam ylläpitää jokaiselle parametrille ensimmäisen ja toisen momentin estimaatit, minkä vuoksi optimointitila voi kuluttaa huomattavan määrän muistia. Tämän vuoksi on kehitetty muistitehokkaampia adaptiivisia menetelmiä, kuten Adafactor, joissa pyritään säilyttämään adaptiivisen skaalaamisen keskeiset edut pienemmällä muistinkulutuksella [38].

Tässä tutkielmassa tarkastelun painopiste rajataan AdaGrad-, RMSProp- ja Adam-tyyppisiin menetelmiin, sillä ne muodostavat historiallisesti ja käsitteellisesti keskeisen perustan myöhemmille adaptiivisille optimointialgoritmeille. Tutkielman empiirisessä osassa arvioitavan järjestelmän parametrit ovat kiinteät, eikä sen koulutuksessa käytettyjä menetelmiä tunneta. Optimoinnin tarkastelu ei siten selitä yksittäisiä havaintoja, vaan perustelee, miksi kielimallin käyttäytymistä on tarkoituksenmukaista arvioida tilastollisena ilmiönä. Seuraavassa luvussa siirrytään arkkitehtuuriin, jonka parametreja edellä kuvatuilla menetelmillä optimoidaan.

5 Dekooderipohjainen Transformer-arkkitehtuuri suurille kielimalleille

Luvuissa 2 ja 3 kielimallinnusta tarkasteltiin seuraavan tokenin ehdollisen todennäköisyyden estimointina. Olkoon $\mathcal{V}$ tokenisanasto ja $\Delta(\mathcal{V})$ sanaston yli määriteltyjen todennäköisyysjakaumien joukko. Kielimalli voidaan tällöin kuvata funktiona, joka muuntaa aiemmin havaitun tokenikontekstin seuraavan tokenin todennäköisyysjakaumaksi.

n-gram-malli ennustaa tokenin $n-1$ mittaisen kontekstin perusteella. Se voidaan siten kuvata funktiona

$$f_{\text{ngram}} \colon \mathcal{V}^{n-1} \to \Delta(\mathcal{V}).$$

Luvussa 3 tarkastellussa MLP-kielimallissa syötekerroksen dimensio sitoo mallin tiettyyn kontekstin pituuteen. Transformer-mallit perustuvat samaan ennustustehtävään, mutta tokenien esityksiä käsitellään sekvenssinä, jonka eri positioihin sovelletaan samoja opittavia muunnoksia. Sama parametrisaatio soveltuu näin eripituisiin syötteisiin mallin konteksti-ikkunan rajoissa.

Olkoon L mallin suurin sallittu kontekstinpituus. Oletetaan, että mallin syöte on aina epätyhjä tokenijono (tyhjä syöte korvataan aloitustokenilla $\langle s \rangle$), jolloin dekooderipohjainen Transformer-kielimalli voidaan kuvata funktiona

$$f_{\text{Tr}} \colon \bigcup_{m=1}^{L} \mathcal{V}^m \to \Delta(\mathcal{V}).$$

Malli voi siten muodostaa seuraavan tokenin jakauman millä tahansa enintään L tokenin pituisella kontekstilla. Käytännössä suurin kontekstinpituus on edelleen arkkitehtuurin ja laskentaresurssien rajoittama, mutta mallin parametrien ei tarvitse olla sidottuja yhteen tiettyyn kontekstipituuteen.

Tässä luvussa tarkastellaan erityisesti dekooderipohjaista Transformer-arkkitehtuuria, joka muodostaa useiden generatiivisten kielimallien, kuten GPT-malliperheen, matemaattisen perustan [29]. Transformer-arkkitehtuurin esittelivät alun perin Vaswani ym. vuonna 2017 [43]. Alkuperäinen arkkitehtuuri oli konekäännökseen suunniteltu enkooderi–dekooderi-rakenne. Pian tämän jälkeen esitettiin, että pelkkään Transformer-dekooderiin perustuva autoregressiivinen kielimalli voidaan esikouluttaa suurilla tekstiaineistoilla ja hienosäätää useisiin kielenkäsittelytehtäviin [29].

Arkkitehtuurin tarkastelu palvelee tutkielman empiiristä kysymyksenasettelua. Kun kielimalli ymmärretään kontekstista seuraavan tokenin jakaumaan johtavana kuvauksena, voidaan täsmentää, mistä samaa tiedontarvetta koskevien SQL-kyselyiden vaihtelu voi aiheutua. Vaihtelu voi syntyä generoinnin valintasäännöstä tai siitä, että merkitykseltään sama kysymys tuottaa eri tokenikontekstin ja siten eri ehdollisen jakauman.

5.1 Itsehuomio

Transformer-mallin keskeinen mekanismi on itsehuomio (engl. *self-attention*). Sen avulla tokenien merkitys ei määräydy pelkän etäisyyden perusteella, vaan malli pai-

nottaa dynaamisesti kunkin tokenin kannalta olennaista tietoa koko sekvenssin laajuudelta, kun se päivittää upotusvektoreita.

Text-to-SQL-tehtävässä sama mekanismi mahdollistaa sen, että kysymyksen ilmausten esitykset voivat painottaa kontekstiin sisällytettyjä taulujen, sarakkeiden ja käsitteiden kuvauksia. Skeeman ja kysymyksen välinen vastaavuus muodostuu näin osana mallin kontekstualisoituja esityksiä eikä erillisenä sääntönä.

Olkoon tarkasteltavan tokenijonon pituus T ja olkoon $h_i \in \mathbb{R}^{1\times d}$ position i tokenia vastaava esitysvektori. Näistä muodostetaan syötematriisi

$$H = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_T \end{bmatrix} \in \mathbb{R}^{T\times d}.$$

Itsehuomiomekanismi muodostaa syöte-esityksistä kolme erillistä lineaarista projektiota: kyselyn (engl. *query*), avaimen (engl. *key*) ja arvon (engl. *value*). Olkoot $W_Q, W_K \in \mathbb{R}^{d\times d_k}$ ja $W_V \in \mathbb{R}^{d\times d_v}$ opittavia painomatriiseja. Kysely-, avain- ja arvomatriisit lasketaan kertomalla syötematriisi näillä painoilla

$$\begin{aligned} Q &= HW_Q \in \mathbb{R}^{T\times d_k}, \\ K &= HW_K \in \mathbb{R}^{T\times d_k}, \\ V &= HW_V \in \mathbb{R}^{T\times d_v}. \end{aligned}$$

Kyselymatriisi määrittää haettavan informaation tyypin kunkin tokenin kohdalla, kun taas avainmatriisi kuvaa tokenin tarjoamaa kontekstitietoa. Arvomatriisi puolestaan edustaa varsinaista informaatiosisältöä, josta uudet kontekstualisoidut esitykset muodostetaan huomiopainotusten avulla.

Kyselyiden ja avainten välinen vastaavuus lasketaan pistetuloina, joista muodostuu skaalattu huomiopistematriisi

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{T\times T}.$$

Matriisin alkio S_{ij} mittaa position i kyselyn ja position j avaimen yhteensopivuutta. Skaalausta voidaan perustella tarkastelemalla kysely- ja avainvektoreita, joiden komponentit ovat riippumattomia, nollakeskiarvoisia ja yksikkövarianssisia. Näillä oletuksilla niiden pistetulon varianssi on d_k, joten jakaminen luvulla $\sqrt{d_k}$ palauttaa varianssin arvoon yksi. Vaikka oletukset eivät yleisesti toteudu täsmällisesti opituilla esityksillä, skaalaus hillitsee huomiopisteiden suuruuden kasvua dimension mukana. Tämä vähentää softmax-funktion saturoitumisen riskiä ja tukee vakaampaa gradienttipohjaista koulutusta [43].

Huomiopisteet normalisoidaan todennäköisyysjakaumiksi soveltamalla softmax-funktiota riveittäin, jolloin saadaan huomiopainomatriisi

$$A = \mathrm{softmax}(S) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{T\times T},$$

missä kukin alkio $A_{ij} \in [0,1]$ ilmaisee, kuinka suuri painoarvo position j arvolle annetaan position i uutta esitystä laskettaessa, ja riveille pätee $\sum_{j=1}^{T} A_{ij} = 1$.

Lopullinen itsehuomion tulos lasketaan arvomatriisin rivien painotettuna summana matriisitulon avulla

$$\text{Attention}(H) = AV = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \in \mathbb{R}^{T\times d_v}.$$

Tuloksena saadun matriisin kukin rivi i edustaa position i uutta kontekstualisoitua esitystä, johon on koottu tietoa kaikista sekvenssin tokeneista huomiopainojen A_{ij} mukaisesti.

Autoregressiivisissä kielimalleissa tavoitteena on mallintaa seuraavan tokenin ehdollista todennäköisyyttä, joten tokenin esitys ei voi nojata tulevaan kontekstiin. Tätä varten itsehuomiota rajoitetaan kausaalisella maskilla (engl. *causal mask*), joka estää informaation vuotamisen tulevaisuudesta asettamalla position i huomiopainot nollaksi kaikille positioille $j > i$. Tämä toteutetaan lisäämällä huomiopisteisiin ennen softmax-funktiota maskimatriisi $M \in (\mathbb{R}\cup\{-\infty\})^{T\times T}$, jonka alkiot määritellään muodossa

$$M_{ij} = \begin{cases} 0, & \text{kun } j \le i, \\ -\infty, & \text{kun } j > i. \end{cases}$$

Kausaalinen itsehuomio lasketaan tällöin muodossa

$$\text{CausalAttention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

Käytännöllä $\exp(-\infty) = 0$ huomiopainomatriisista muodostuu alakolmiomatriisi. Tällöin kunkin tokenin esitys riippuu vain siitä itsestään ja sitä edeltävistä tokeneista, mikä säilyttää generointiprosessin kausaalisen rakenteen.

Yksi huomiopää kokoaa informaatiota yhden opitun kysely-, avain- ja arvoprojektion varassa, ja sama huomiopainotus A_{ij} koskee arvovektorien kaikkia koordinaatteja. Monipäinen itsehuomio (engl. *multi-head attention*) mahdollistaa useiden erilaisten painotusten ja esitysten muodostamisen rinnakkain [43].

Olkoon päiden lukumäärä n_{heads}. Kukin pää $k \in \{1,\dots,n_{\text{heads}}\}$ suorittaa kausaalisen itsehuomion omilla opittavilla projektioillaan

$$\text{head}_k = \text{CausalAttention}\left(HW_k^Q,\, HW_k^K,\, HW_k^V\right),$$

missä $W_k^Q, W_k^K \in \mathbb{R}^{d\times d_k}$ ja $W_k^V \in \mathbb{R}^{d\times d_v}$. Kukin pää muodostaa näin oman huomiopainomatriisinsa ja omat painotetut summansa arvovektoreista. Päät voivat oppia toisiaan täydentäviä riippuvuuksia. Esimerkiksi yksi pää saattaa keskittyä syntaktisiin rakenteisiin ja toinen laajempiin semanttisiin yhteyksiin. Todellisuudessa opitut suhteet eivät kuitenkaan jakaudu näin siististi, vaan ne ovat usein huomattavasti monimutkaisempia ja vaikeammin tulkittavia [9].

Lopuksi eri päiden tuottamat esitykset ketjutetaan ja yhdistetään lineaarisella projektiolla takaisin mallin ulottuvuuteen d

$$\text{CausalMultiHead}(H) = [\text{head}_1 \parallel \text{head}_2 \parallel \dots \parallel \text{head}_{n_{\text{heads}}}]\, W^O,$$

missä $\parallel$ merkitsee sarakkeittaista ketjuttamista ja $W^O \in \mathbb{R}^{n_{\text{heads}}d_v\times d}$ on opittava parametrimatriisi.

Monipäisyyden etu on, että eri päät voivat käyttää toisistaan poikkeavia huomiopainotuksia ja arvoprojektioita, joiden tuottama informaatio yhdistetään tulosprojektiossa. Käytännössä pään dimensioiksi valitaan usein $d_k = d_v = d/n_{\text{heads}}$, jolloin monipäisen itsehuomion laskennallinen vaativuus vastaa karkeasti täyden dimension yksipäistä huomiota [43].

5.2 Positiokoodaus

Maskiton itsehuomio on permutaatioekvivariantti eli ilman erillistä positiotietoa se ei kykene erottelemaan tokenien järjestystä. Olkoon $P \in \mathbb{R}^{T \times T}$ permutaatiomatriisi, joka järjestää syötematriisin H rivit uudelleen. Koska kysely-, avain- ja arvomatriisit muodostetaan riveittäin samoilla lineaarisilla kuvauksilla, permutoidusta syötteestä saadaan $Q' = PQ$, $K' = PK$ ja $V' = PV$. Tällöin huomiopistematriisille pätee

$$\frac{Q'(K')^\top}{\sqrt{d_k}} = P\frac{QK^\top}{\sqrt{d_k}}P^\top.$$

Riveittäin sovellettava softmax-funktio säilyttää vastaavan permutaation, joten maskittomalle itsehuomiolle pätee

$$\text{Attention}(PH) = P\,\text{Attention}(H).$$

Syötteen tokenien permutointi siis ainoastaan permutoi tulosteet samalla tavalla. Maskiton itsehuomiomekanismi ei sellaisenaan sisällä tietoa siitä, missä järjestyksessä tokenit esiintyvät. Autoregressiivisessa mallissa kausaalinen maski rikkoo tämän ominaisuuden mielivaltaisten permutaatioiden suhteen, sillä se määrää, mitkä positiot ovat kunkin tokenin menneisyydessä. Kausaalinen maski tuo siten malliin järjestysrakennetta, jonka avulla malli voi myös oppia hyödyntämään sijaintiin liittyvää tietoa. Maski ei kuitenkaan sisällytä kyselyn ja avaimen väliseen yhteensopivuuteen eksplisiittistä tietoa siitä, sijaitseeko avain esimerkiksi yhden vai usean tokenin päässä kyselystä. Erillinen positiokoodaus antaa mallille suoran mekanismin tokenien sijaintien tai niiden välisten positioerojen esittämiseen.

Positiotieto voidaan sisällyttää Transformer-arkkitehtuuriin usealla tavalla. Alkuperäisessä Transformerissa tokeniesityksiin lisätään absoluuttiset, sinimuotoiset positiokoodaukset [43]. Suhteellisissa menetelmissä huomiopisteitä puolestaan muokataan positioiden välisen etäisyyden perusteella. Monissa moderneissa dekooderipohjaisissa kielimalleissa käytetään kiertävää positiokoodausta (engl. *rotary positional embedding*, RoPE), jossa positiotieto liitetään kysely- ja avainvektoreihin rotaatioiden avulla [40].

Olkoon yhden huomiopään kysely- ja avainvektorien dimensio d_k parillinen. RoPE jakaa vektorien koordinaatit $d_k/2$ kaksiulotteiseen pariin ja kiertää jokaista paria position mukaisella kulmalla. Kun parit indeksoidaan arvoilla $\ell \in \{0, \ldots, d_k/2 - 1\}$, tavanomainen kulmataajuus pareille on

$$\omega_\ell = 10000^{-2\ell/d_k}.$$

Kulmaa ϕ vastaava kaksiulotteinen rotaatiomatriisi on

$$R(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix}.$$

Position i koko rotaatiomatriisi muodostetaan näistä kaksiulotteisista rotaatioista lohkodiagonaalisena matriisina

$$R_i = \operatorname{diag}\left(R(i\omega_0), R(i\omega_1), \ldots, R\left(i\omega_{d_k/2-1}\right)\right) \in \mathbb{R}^{d_k \times d_k}.$$

Olkoon $q_i = Q_{i,:} \in \mathbb{R}^{1\times d_k}$ position i kyselyvektori ja $k_j = K_{j,:} \in \mathbb{R}^{1\times d_k}$ position j avainvektori. RoPE korvaa nämä kierretyillä rivivektoreilla

$$\tilde{q}_i = q_i R_i, \qquad \tilde{k}_j = k_j R_j.$$

Arvovektoreihin rotaatiota ei tavallisesti sovelleta. Kierrettyjen kysely- ja avainvektorien pistetuloksi saadaan

$$\tilde{q}_i \tilde{k}_j^\top = q_i R_i (k_j R_j)^\top = q_i R_i R_j^\top k_j^\top.$$

Kaksiulotteisille rotaatiomatriiseille pätee $R(\alpha)R(\beta)^\top = R(\alpha - \beta)$. Sama ominaisuus pätee lohkoittain myös RoPE-rotaatioille, joten

$$R_i R_j^\top = R_{i-j}.$$

Siten RoPE-huomiopiste voidaan kirjoittaa muodossa

$$\tilde{q}_i \tilde{k}_j^\top = q_i R_{i-j} k_j^\top.$$

Vaikka kysely ja avain kierretään niiden absoluuttisten positioiden perusteella, rotaatioiden yhteisvaikutus pistetulossa riippuu ainoastaan positioiden suhteellisesta siirtymästä $i - j$.

Kun kierretyt kyselyt ja avaimet kootaan matriiseiksi $\tilde{Q}$ ja $\tilde{K}$, kausaalinen itsehuomio saa muodon

$$\text{CausalAttention}\left(\tilde{Q}, \tilde{K}, V\right) = \text{softmax}\left(\frac{\tilde{Q}\tilde{K}^\top}{\sqrt{d_k}} + M\right) V.$$

Koska R_i on ortogonaalinen matriisi, rotaatio säilyttää kysely- ja avainvektorien normit. Eri koordinaattipareille määritellyt kulmataajuudet määräävät, kuinka nopeasti rotaatiokulma muuttuu position funktiona. Suurilla taajuuksilla kulma muuttuu nopeasti position kasvaessa, kun taas pienillä taajuuksilla muutos on hitaampaa. Näin eri koordinaattiparit kuvaavat positioeroja eri mittakaavoilla. RoPE ei muuta itsehuomion asymptoottista laskennallista vaativuutta [40].

5.3 Dekooderikerros

Yksittäinen dekooderikerros koostuu kausaalisesta monipäisestä itsehuomiosta ja positiokohtaisesta myötäkytkentäverkosta (engl. *feed-forward network*, FFN). Kumpaankin komponenttiin liitetään jäännöskytkentä (engl. *residual connection*) sekä

normalisointi. Normalisointi auttaa hallitsemaan esitysten mittakaavaa, ja jäännöskytkennät helpottavat gradienttien kulkua syvässä verkossa. Yhdessä nämä rakenteet tukevat koulutuksen vakautta [14, 2, 43].

Laskennan vakauttamiseksi esitykset normalisoidaan ennen kuhunkin alilohkoon syöttämistä. Normalisointifunktio $\mathrm{Norm}\colon \mathbb{R}^{1\times d} \to \mathbb{R}^{1\times d}$ sovelletaan itsenäisesti kunkin position esitysvektoriin. Yleinen valinta nykyaikaisissa malleissa on neliöllisen keskiarvon normalisointi (engl. *root mean square normalization*, RMSNorm) [50]. Tämä skaalaa vektorin sen komponenttien neliöllisen keskiarvon avulla

$$\mathrm{RMSNorm}(x) = \frac{x}{\mathrm{RMS}(x)} \odot \gamma,$$

missä $\odot$ on Hadamard-tulo, $\gamma \in \mathbb{R}^{1\times d}$ on opittava skaalausparametri ja

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \epsilon},$$

missä $\epsilon > 0$ estää nollalla jakamisen. Matriisille $H \in \mathbb{R}^{T\times d}$ normalisointi $\mathrm{Norm}(H)$ määritellään soveltamalla kuvausta riveittäin jokaiseen esitysvektoriin.

Itsehuomiokerrosta seuraa positiokohtainen myötäkytkentäverkko. Toisin kuin luvussa 3 tarkastellussa MLP-kielimallissa, jossa verkko muuntaa koko kontekstin suoraan seuraavan tokenin logiteiksi, Transformer-lohkossa FFN toimii positiokohtaisena muunnoksena. Sitä sovelletaan itsenäisesti ja samoilla parametreilla kunkin position esitykseen erikseen. Positioiden välisestä informaation siirrosta vastaa siten itsehuomio, kun taas FFN vastaa yksittäisen esityksen rikastamisesta ja epälineaarisesta muuntamisesta.

Kaksi affiinia muunnosta sisältävä FFN voidaan määritellä vektorille $x \in \mathbb{R}^{1\times d}$ muodossa

$$\mathrm{FFN}(x) = \phi(xW_1 + b_1)W_2 + b_2,$$

missä $W_1 \in \mathbb{R}^{d\times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff}\times d}$, $b_1 \in \mathbb{R}^{1\times d_{ff}}$, $b_2 \in \mathbb{R}^{1\times d}$ ja ϕ on koordinaateittain sovellettava epälineaarinen aktivaatiofunktio. Sisäisen piiloesityksen dimensio d_{ff} valitaan tyypillisesti suuremmaksi kuin mallin dimensio d. Alkuperäisessä Transformer-arkkitehtuurissa käytettiin valintaa $d_{ff} = 4d$ [43].

Kun tokenien esitykset ovat matriisin $H \in \mathbb{R}^{T\times d}$ riveinä, sama laskenta kirjoitetaan muodossa

$$\mathrm{FFN}(H) = \phi\left(HW_1 + \mathbf{1}b_1\right)W_2 + \mathbf{1}b_2,$$

missä $\mathbf{1} \in \mathbb{R}^{T\times 1}$ on sarakeykkösvektori, jonka avulla bias-vektorit lisätään jokaiselle riville.

Koko dekooderilohko voidaan määritellä funktiona $\mathrm{DecoderBlock}\colon \mathbb{R}^{T\times d} \to \mathbb{R}^{T\times d}$. Olkoon syöte $H \in \mathbb{R}^{T\times d}$. Lohkon laskenta etenee kahdessa vaiheessa

$$H^{(1)} = H + \mathrm{CausalMultiHead}(\mathrm{Norm}_1(H)),$$

$$\mathrm{DecoderBlock}(H) = H^{(1)} + \mathrm{FFN}(\mathrm{Norm}_2(H^{(1)})),$$

missä Norm_1 ja Norm_2 ovat toisistaan erillisiä normalisointeja omilla opittavilla parametreillaan. Normalisointi sijoitetaan ennen kumpaakin alilohkoa, joten rakenne noudattaa niin kutsuttua *pre-norm*-järjestystä [47].

Jäännöskytkentä muodostaa gradientille suoran kulkureitin alilohkon ohi, ja normalisointi hillitsee esitysten mittakaavan kasvua kerrosten määrän kasvaessa, mikä helpottaa syvien verkkojen optimointia [14, 47].

5.4 Dekooderikerroksista seuraavan tokenin jakaumaan

Yksittäinen dekooderilohko muuntaa tokenien esitysmatriisin toiseksi samankokoiseksi esitysmatriisiksi. Kokonaisessa kielimallissa tokenien upotuksesta saatua esitysmatriisia käsitellään peräkkäisissä dekooderilohkoissa, minkä jälkeen tuloskerros muuntaa lopulliset esitykset seuraavan tokenin todennäköisyysjakaumaksi seuraavalle tokenille.

Olkoon syötteenä havaittu konteksti $w_{1:T} = (w_1, \ldots, w_T)$, missä $1 \leq T \leq L$. Kuten luvussa 3 määriteltiin, kukin token muunnetaan esitysvektoriksi upotusmatriisin $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ avulla. Ensimmäisen lohkon syötematriisi $H^{[0]} \in \mathbb{R}^{T \times d}$ muodostetaan näistä riveittäin

$$H^{[0]}_{i,:} = e(w_i) = x(w_i)E, \qquad i = 1, \ldots, T.$$

Tässä tarkasteltavassa arkkitehtuurissa positiotieto sisällytetään itsehuomion ky -sely- ja avainvektoreihin RoPE-rotaatioiden avulla. Jatkossa dekooderilohkon kausaalisen monipäisen itsehuomion oletetaan sisältävän edellä määritellyt rotaatiot kunkin huomiopään kyselyille ja avaimille.

Olkoon dekooderilohkojen lukumäärä B. Esitysmatriisia muunnetaan peräkkäisissä lohkoissa seuraavasti

$$H^{[b]} = \mathrm{DecoderBlock}_b\left(H^{[b-1]}\right), \qquad b = 1, \ldots, B.$$

Kullakin lohkolla on omat opittavat parametrinsa, mutta esitysmatriisin koko säilyy kaikissa vaiheissa samana. Hakasulkeinen yläindeksi ilmaisee tässä koko mallin dekooderilohkon indeksin, eikä yksittäisen lohkon sisäistä laskentavaihetta.

Kausaalisuus säilyy myös useita lohkoja yhdistettäessä. Ensimmäisessä lohkossa position i esitys voi hyödyntää ainoastaan positioita $1, \ldots, i$. Seuraavassa lohkossa itsehuomio yhdistää vain näihin positioihin kuuluvia esityksiä, joista yksikään ei sisällä tietoa positiota i seuraavista tokeneista. Koska normalisointi ja myötäkytkentäverkko toimivat positiokohtaisesti, ne eivät muuta tätä riippuvuusrakennetta. Siten position i esitys riippuu kaikissa lohkoissa vain kontekstista $w_{1:i}$.

Tarkasteltavassa pre-norm-rakenteessa viimeisen dekooderilohkon tulokseen sovelletaan positiokohtaista normalisointia

$$Z = \mathrm{Norm}_{\mathrm{out}}\left(H^{[B]}\right) \in \mathbb{R}^{T \times d}.$$

Olkoon $z_i = Z_{i,:} \in \mathbb{R}^{1 \times d}$ position i lopullinen esitys vektorina. Se muunnetaan sanaston ulottuvuuteen affiinilla kuvauksella

$$\ell_i = z_i W_{\mathrm{out}} + b_{\mathrm{out}},$$

missä $W_{\mathrm{out}} \in \mathbb{R}^{d \times |\mathcal{V}|}$ ja $b_{\mathrm{out}} \in \mathbb{R}^{1 \times |\mathcal{V}|}$ ovat opittavia parametreja. Vektorin ℓ_i komponentti $\ell_{i,j}$ vastaa sanaston tokenin v_j normalisoimatonta pistemäärää eli logittia. Näistä muodostetaan todennäköisyysjakauma softmax-funktiolla.

Merkitään mallin kaikkia opittavia parametreja symbolilla θ. Position i piiloesitys tiivistää informaation kontekstista $w_{1:i}$, joten sen tuottama logittivektori ℓ_i määrittää todennäköisyysjakauman seuraavalle tokenille w_{i+1}

$$p_\theta(w_{i+1} = v_j \mid w_{1:i}) = \frac{\exp(\ell_{i,j})}{\sum_{k=1}^{|\mathcal{V}|} \exp(\ell_{i,k})}, \qquad v_j \in \mathcal{V}.$$

Koko syötekontekstia $w_{1:T}$ seuraavan uuden tokenin w_{T+1} jakauma saadaan siten suoraan viimeisen position logiteista

$$f_{\mathrm{Tr}}(w_{1:T}) = \mathrm{softmax}(\ell_T) \in \Delta(\mathcal{V}).$$

Kielimallin laskentaketju etenee näin diskreeteistä tokeneista jatkuviin esityksiin, kontekstuaalisiin muunnoksiin ja lopulta takaisin diskreetin sanaston yli määritellyksi jakaumaksi.

Arkkitehtuuri määrittää seuraavan tokenin jakauman annetuilla parametreilla θ. Seuraavaksi tarkastellaan, miten parametrit opitaan aineistosta ja miten opittua jakaumaa käytetään kokonaisten tokenijonojen muodostamiseen.

5.5 Autoregressiivinen koulutus ja generointi

Transformer-kielimallin koulutus perustuu samaan uskottavuusperiaatteeseen kuin luvussa 3 tarkastellussa MLP-kielimallissa. Todennäköisyyksien ketjusäännön perusteella tokenijonon $w_{1:T}$ todennäköisyys voidaan esittää muodossa

$$p_\theta(w_{1:T}) = \prod_{t=1}^{T} p_\theta(w_t \mid w_{<t}),$$

missä $w_{<t} = (w_1, \ldots, w_{t-1})$.

Olkoon koulutusaineisto $\mathcal{D} = \{w^{(1)}, \ldots, w^{(N)}\}$, missä $w^{(r)} = (w_1^{(r)}, \ldots, w_{T_r}^{(r)})$ ja $1 \le T_r \le L$. Aineiston tokenikohtainen keskimääräinen negatiivinen log-uskottavuus on

$$J(\theta; \mathcal{D}) = -\frac{1}{\sum_{r=1}^{N} T_r} \sum_{r=1}^{N} \sum_{t=1}^{T_r} \log p_\theta\left(w_t^{(r)} \mid w_{<t}^{(r)}\right),$$

ja mallin koulutus esitetään optimointiongelmana

$$\min_\theta J(\theta; \mathcal{D}).$$

Toisin kuin n-gram-mallissa, eri kontekstien ehdolliset jakaumat eivät ole itsenäisiä parametreja, vaan ne muodostetaan yhteisellä parametrisaatiolla θ. Optimointiongelma ei siten hajoa erillisiksi kontekstikohtaisiksi tehtäviksi, eikä sille yleensä ole suljetun muodon ratkaisua.

Häviö voidaan esittää myös tokenikohtaisten tavoitejakaumien ja mallin ennustejakaumien ristientropioiden keskiarvona. Merkitään position t tavoitejakaumaa ja mallin ennustejakaumaa komponenteittain

$$q_{t,j}^{(r)} = \mathbf{1}\{w_t^{(r)} = v_j\}, \qquad p_{\theta,t,j}^{(r)} = p_\theta\left(v_j \mid w_{<t}^{(r)}\right),$$

missä $\mathbf{1}\{\cdot\}$ on indikaattorifunktio. Koska tavoitejakauma keskittää kaiken todennäköisyysmassan havaittuun tokeniin, saadaan

$$-\sum_{j=1}^{|\mathcal{V}|} q_{t,j}^{(r)} \log p_{\theta,t,j}^{(r)} = -\log p_\theta\left(w_t^{(r)} \mid w_{<t}^{(r)}\right).$$

Negatiivisen log-uskottavuuden minimointi on täten yhtäpitävää tavoitejakaumien ja ennustejakaumien keskimääräisen ristientropian minimoinnin kanssa.

Koulutuksessa ehdollistetaan aineistossa havaittuihin prefikseihin. Sekvenssiä $w_{1:T}$ vastaava syöte ja tavoitejono ovat

$$x = (\langle s \rangle, w_1, \ldots, w_T), \qquad y = (w_1, \ldots, w_T, \langle /s \rangle).$$

Kun ℓ_t on tämän syötejonon position t logittivektori, kausaalinen maski takaa, että

$$\operatorname{softmax}(\ell_t)_j = p_\theta(w_t = v_j \mid w_{<t}).$$

Kaikkien positioiden ennusteet voidaan siten laskea rinnakkain yhdellä eteenpäinlaskennalla.

Softmax-funktion ja ristientropian yhdistelmä antaa position häviölle yksinkertaisen derivaatan logittien suhteen

$$\frac{\partial}{\partial \ell_{t,j}^{(r)}} \left(-\sum_{k=1}^{|\mathcal{V}|} q_{t,k}^{(r)} \log p_{\theta,t,k}^{(r)} \right) = p_{\theta,t,j}^{(r)} - q_{t,j}^{(r)}.$$

Parametrien suhteen gradientti saadaan ketjusäännöllä, ja se lasketaan vastavirta-algoritmilla, jolloin parametreja voidaan päivittää luvussa 4 käsitellyllä gradienttimenetelmällä.

Laajalla tekstiaineistolla toteutettua koulutusta kutsutaan autoregressiiviseksi esikoulutukseksi [29]. Esikoulutettua mallia voidaan edelleen hienosäätää esimerkiksi ohjeiden noudattamiseen ohjevirityksellä [27]. Valvotussa ohjevirityksessä havainto koostuu syötekontekstista c ja tavoitevastauksesta $y = (y_1, \ldots, y_K)$, jolloin vastaustokeneihin rajattu negatiivinen log-uskottavuus on

$$-\log p_\theta(y \mid c) = -\sum_{t=1}^{K} \log p_\theta(y_t \mid c, y_{<t}).$$

Optimointiperiaate säilyy samana, mutta häviö kohdistetaan annetun kontekstin ehdollistamaan tavoitevastaukseen.

Kun koulutettua mallia sovelletaan tekstin generointiin, sen parametrit pidetään kiinteinä. Malli muodostaa seuraavan tokenin todennäköisyysjakauman nykyisen kontekstin perusteella. Jakaumasta valitaan tokeni, joka liitetään kontekstin loppuun, ja laskenta toistetaan näin täydentyneellä kontekstilla.

Generointi päättyy esimerkiksi lopetustokeniin tai tuotokselle asetettuun pituusrajaan. Myös syötteen ja tuotettujen tokenien yhteinen kontekstinpituus rajoittaa laskentaa. Rajan saavuttamisen jälkeen jatkaminen edellyttää kontekstin lyhentämistä tai muuta erikseen määriteltyä kontekstinhallintaa.

Yksinkertainen valintasääntö generoitavalle tokenille on

$$\widehat{w}_{t+1} = \arg\max_{v \in \mathcal{V}} p_\theta(v \mid w_{1:t}),$$

joka valitsee jokaisessa vaiheessa kyseisen jakauman todennäköisimmän tokenin. Vaihtoehtoisesti voidaan käyttää otantaa, jossa seuraava tokeni arvotaan jakaumasta. Otannassa jakauman keskittyneisyyttä voidaan muuttaa kaavalla

$$p_{\theta,\tau}(v \mid w_{1:t}) = \frac{\exp(\ell_{t,v}/\tau)}{\sum_{u \in \mathcal{V}} \exp(\ell_{t,u}/\tau)},$$

missä $\tau > 0$ on lämpötilaparametri. Kun $\tau = 1$, saadaan alkuperäinen softmax-jakauma. Pienempi lämpötila keskittää massaa suurimpien logittien kohdalle, kun taas suurempi lämpötila tasoittaa jakaumaa.

Valintasäännöllä on suora yhteys tuotosten toistettavuuteen. Ahne valinta on kiinteällä kontekstilla ja parametreilla periaatteessa deterministinen, kun taas otannassa saman kontekstin toistot voivat tuottaa eri tokenijonoja. Koska generointi etenee autoregressiivisesti, jo yksittäisen varhaisen tokenin ero muuttaa kaikkien myöhempien jakaumien ehdollistavaa kontekstia. Käytännön järjestelmissä myös laskennan numeeriset erot ja palvelun mahdolliset päivitykset voivat aiheuttaa vaihtelua, vaikka syöte pysyisi samana.

Kielimallien sovelluksissa seuraavan tokenin ennustaminen toimii mekanismina laajempien tehtävien ratkaisemiseen. Syötekontekstissa esitetään esimerkiksi kysymys tai ohje, johon mallin odotetaan tuottavan sisällöllisesti oikea vastaus. Tällöin on erotettava toisistaan tokenijonon todennäköisyys mallin määrittämässä jakaumassa ja sen oikeellisuus annetun tehtävän ratkaisuna. Todennäköisin tokenijono ei välttämättä ratkaise tehtävää oikein, sillä autoregressiivinen koulutustavoite ei sellaisenaan takaa tuotoksen loogista tai toiminnallista oikeellisuutta.

Tämä ero korostuu luonnollisella kielellä esitettyjen kysymysten muuntamisessa SQL-kyselyiksi. Generoitu SQL-kysely voi olla syntaktisesti kelvollinen ja suoritettava, vaikka se vastaisi eri kysymykseen kuin käyttäjä tarkoitti.

5.6 Kielimalli Text-to-SQL-tehtävissä

Text-to-SQL-tehtävässä luonnollisella kielellä esitetty tiedontarve muunnetaan tietokantaan kohdistuvaksi SQL-kyselyksi. Autoregressiivisen kielimallin näkökulmasta kyseessä on ehdollinen generointitehtävä. Tuotettavan tokenijonon tulee muodostaa SQL-kysely, jonka merkitys vastaa käyttäjän kysymystä käytettävissä olevan tietokannan ja sitä koskevien määritelmien puitteissa.

Olkoon q luonnollisen kielen kysymys, s tietokannan skeeman kuvaus ja m semanttinen lisätieto. Skeema kuvaa esimerkiksi taulut, sarakkeet ja niiden väliset suhteet. Semanttinen lisätieto täydentää tätä kuvausta käsitteiden merkityksillä, mittarien määritelmillä ja muilla tulkintasäännöillä.

Nämä tiedot esitetään mallille tokenikontekstina

$$c = \text{Serialize}(q, s, m),$$

missä Serialize tarkoittaa tietojen muuntamista mallin käsittelemäksi tokenijonoksi. Konteksti voi sisältää esimerkiksi tehtäväohjeen, skeeman tekstimuotoisen kuvauksen ja esimerkkejä kysymysten ja SQL-kyselyiden vastaavuuksista.

Kun SQL-kyselyn tokenijono on $y = (y_1, \ldots, y_K)$, sen ehdollinen todennäköisyys voidaan kirjoittaa muodossa

$$p_\theta(y \mid c) = \prod_{t=1}^{K} p_\theta(y_t \mid c, y_{<t}).$$

Tämä esitys täsmentää kaksi tutkielman kannalta keskeistä vaihtelun lähdettä. Saman kysymyksen toistoissa konteksti c pysyy samana, joten tuotosten erot johtuvat generoinnin valintasäännöstä tai järjestelmän muista epädeterministisistä tekijöistä. Jos kysymys q korvataan merkitykseltään samalla muotoilulla q', muodostuu uusi konteksti $c' = \text{Serialize}(q', s, m)$. Mallin kannalta c ja c' ovat eri tokenijonoja, eikä jakaumien $p_\theta(\cdot \mid c)$ ja $p_\theta(\cdot \mid c')$ yhtäpitävyys seuraa arkkitehtuurista tai koulutustavoitteesta. Kielellisten variaatioiden suhteen robusti järjestelmä tuottaisi molemmista konteksteista merkitykseltään yhtäpitäviä kyselyitä, vaikka niiden tokenijonot eroaisivat toisistaan.

Todennäköisyysmalli $p_\theta(y \mid c)$ kuvaa kuitenkin vain kielimallin generointivaihetta, ei välttämättä koko Text-to-SQL-järjestelmän toimintaa. Järjestelmä voi myös valita skeemasta relevantit osat, hakea esimerkkejä, tarkistaa tuotetun SQL-kyselyn tai muodostaa korjatun ehdotuksen uudella mallikutsulla [15]. Tällöin sovelluksen toiminta on useiden käsittelyvaiheiden yhteistulos.

Kontekstin ja generoinnissa tarvittavan historian on mahduttava mallin käytettävissä olevaan konteksti-ikkunaan. Laajan tietokannan tapauksessa koko skeeman ja kaiken lisätiedon sisällyttäminen yhteen syötteeseen ei siten välttämättä ole tarkoituksenmukaista.

Keskeinen osatehtävä on skeemaan kohdistaminen (engl. *schema linking*), jossa kysymyksen ilmaukset yhdistetään tietokannan tauluihin, sarakkeisiin ja muihin rakenteisiin [44]. Kysymyksessä käytetty sanasto ei välttämättä vastaa suoraan tietokannan nimiä. Lisäksi oikea vastaus voi edellyttää useiden taulujen yhdistämistä, vaikka käyttäjä ei mainitsisi niitä erikseen.

Tarkastellaan esimerkiksi kysymystä *"Mikä oli viime vuoden myynti asiakasryhmittäin?"* Sen muuntaminen SQL-kyselyksi edellyttää vähintään myyntitapahtumia kuvaavan taulun, myyntimäärän sisältävän sarakkeen, ajallisen rajauksen ja asiakasryhmän tunnistamista. Jos asiakasryhmä sijaitsee eri taulussa kuin myyntitapahtumat, on lisäksi valittava oikea liitosavain ja liitospolku. Ryhmittely ja aggregointi on muodostettava siten, että tuloksen tarkkuustaso vastaa kysymystä.

Pelkkä skeeman tunteminen ei aina ratkaise näitä valintoja. Esimerkin sana *myynti* voi tarkoittaa bruttomyyntiä, nettomyyntiä tai laskutettua myyntiä. Samoin on ratkaistava, vähennetäänkö palautukset ja perustuuko ajallinen rajaus tilauspäivään vai laskutuspäivään. Ilmaus *viime vuosi* edellyttää viiteajankohtaa sekä tietoa siitä, tarkoitetaanko kalenterivuotta vai tilikautta.

Skeema ja semanttinen tieto täydentävät siten toisiaan. Skeema määrittää käytettävissä olevat rakenteet, kun taas semanttinen tieto auttaa tulkitsemaan, miten näitä rakenteita tulee käyttää tietyn tiedontarpeen ratkaisemiseen. Semanttiseksi kerrokseksi voidaan kutsua tietokannan rakenteita täydentävää kuvausta, jossa

määritellään esimerkiksi liiketoimintakäsitteitä, mittareita ja hyväksyttyjä taulujen välisiä suhteita [39].

Kielimallille annettu semanttinen kuvaus muuttaa generoinnin ehdollistavaa kontekstia. Se voi vähentää monitulkintaisuutta tarjoamalla eksplisiittisiä määritelmiä sen sijaan, että mallin olisi pääteltävä merkitykset pelkästään sarakkeiden nimistä tai esikoulutuksessa opituista yhteyksistä. Kuvaus ei kuitenkaan takaa oikeaa tulosta. Malli voi soveltaa sitä väärin, ja virheellinen tai ristiriitainen kuvaus voi itsessään johtaa virheelliseen kyselyyn.

SQL-kyselyn oikeellisuudessa on hyödyllistä erottaa kolme tasoa. Syntaktisesti kelvollinen kysely noudattaa käytetyn SQL-murteen kielioppia. Suoritettava kysely voidaan lisäksi suorittaa kyseisessä tietokantaympäristössä. Semanttisesti oikea kysely toteuttaa käyttäjän tarkoittaman tiedonhaun.

Syntaktisesti kelvollinen kysely voi viitata olemattomaan sarakkeeseen ja olla siksi suorituskelvoton. Suoritettava kysely voi puolestaan käyttää väärää aikarajausta tai liitosta ja tuottaa siten väärän vastauksen. Lisäksi suoritettavuuteen vaikuttavat kyselyn ulkopuoliset tekijät, kuten käyttöoikeudet ja ympäristön tila. Suoritusvirhe ei siten aina merkitse generoidun SQL-kyselyn loogista virhettä.

Olkoon D tietokannan tila eli tietokantainstanssi. Merkitään onnistuneesti suoritetun kyselyn tulosta

$$\mathrm{Exec}(y, D).$$

Jos $y^\star$ on kysymykseen laadittu vertailukysely, tuotettua kyselyä voidaan arvioida vertaamalla suoritustuloksia

$$\mathrm{Exec}(y, D) = \mathrm{Exec}(y^\star, D).$$

Vertailussa käytettävä tulosten yhtäsuuruus on määriteltävä tehtävän mukaisesti. Esimerkiksi rivijärjestyksen merkitys riippuu siitä, edellyttääkö kysymys järjestettyä tulosta.

Pelkkä SQL-kyselyiden merkkijonojen yhtäsuuruus on liian rajoittava oikeellisuuden peruste, sillä eri tavoin kirjoitetut kyselyt voivat toteuttaa saman tiedonhaun. Toisaalta myöskään sama suoritustulos yhdellä tietokantainstanssilla ei yleisesti todista kyselyiden semanttista ekvivalenssia [51]. Virheellinen suodatusehto voi esimerkiksi jäädä havaitsematta, jos aineistossa ei ole rivejä, joihin se vaikuttaisi.

Luonnollisen kielen kysymys ei myöskään aina määritä yksikäsitteistä tavoitekyselyä. Jos keskeinen mittari tai aikaväli on määrittelemätön, useat SQL-kyselyt voivat vastata kysymyksen eri tulkintoja. Tällöin tarkentavan kysymyksen esittäminen voi olla asianmukaisempaa kuin yhden tulkinnan valitseminen ilman perustelua [24]. Jos tarvittavaa tietoa ei ole tietokannassa, oikea toiminta voi puolestaan olla perusteltu ilmoitus siitä, ettei kysymykseen voida vastata käytettävissä olevilla tiedoilla.

Autoregressiivinen kielimalli tarjoaa mekanismin SQL-kyselyiden muodostamiseen, mutta sen tokeniennusteisiin perustuva koulutustavoite ei yksin takaa kyselyiden oikeellisuutta [27]. Käytännön onnistuminen riippuu myös skeeman esitystavasta, semanttisista määritelmistä, kysymyksen yksikäsitteisyydestä ja järjestelmän muista käsittelyvaiheista.

Edellä esitetty kehys muodostaa luvussa 6 määriteltävän tutkimusasetelman käsitteellisen perustan. Autoregressiivinen generointi selittää, miksi samaa tiedontar-

vetta koskevat kyselyt voivat vaihdella, ja oikeellisuuden tasojen erottelu perustelee, miksi vaihtelua ja semanttista vastaavuutta on arvioitava erikseen. Teoriaa ei tulkita kuvaukseksi Cortex Analystin sisäisistä toteutusratkaisuista, vaan sen avulla jäsennetään järjestelmän havaittavaa toimintaa.

6 Metodologia

6.1 Tutkimusasetelma

Tutkimus toteutetaan eksploratiivisena pilottitutkimuksena ja empiirisenä tapaustutkimuksena, jossa arvioidaan yhden kielimallipohjaisen järjestelmän toimintaa rajatussa sovelluskontekstissa. Tarkastelu perustuu viiteen valittuun tiedontarpeeseen, niiden toistuviin suorituksiin ja vaihtoehtoisiin kysymysmuotoiluihin. Vaikka aineisto sisältää 250 erillistä suoritusta, tutkimuksen tehtäväkohtainen kattavuus perustuu viiteen tiedontarpeeseen. Toistot ja kysymysvariaatiot kuvaavat näiden tiedontarpeiden sisäistä vaihtelua, eivät järjestelmän suoriutumista 250 erilaisessa analyysitehtävässä.

Tavoitteena ei ole muodostaa kattavaa arviota Cortex Analystin yleisestä suorituskyvystä, vaan tarkastella sen toimintaa valituissa kyselytilanteissa sekä tunnistaa onnistumisia, virhetilanteita ja jatkotutkimuksessa tarkasteltavia ilmiöitä. Tapaustutkimus mahdollistaa näiden ilmiöiden yksityiskohtaisen analyysin todellisessa kaupallisessa ympäristössä.

Tutkimuksen analyysiyksikkönä on yksittäinen käyttäjän luonnollisen kielen kysymys ja sitä vastaava Cortex Analystin tuottama SQL-kysely. Arviointi kohdistuu siihen, vastaako muodostettu kysely käyttäjän tarkoitusta, hyödyntääkö se oikeita tauluja ja kenttiä sekä noudattaako se tietokannan rakennetta. Näin tutkimus keskittyy suoraan siihen vaiheeseen, jossa kielimallin tulkinta muuttuu koneellisesti suoritettavaksi tietokantakyselyksi.

6.2 Tutkimusaineisto

Tutkimuksen aineistona toimii puhelinkeskusdataa sisältävä tietokanta, joka noudattaa kuvassa 1 esitettyä rakennetta. Tarkastelun keskiössä on yksi keskusteludataa sisältävä päätason taulu, jota täydentävät kolme liitännäistaulua. Liitännäistaulujen avulla voidaan tarkentaa päätason taulun yksittäisten kenttien sisältöä ja merkitystä. Tietokannan rakenne on siten riittävän monipuolinen, jotta voidaan tarkastella sekä yksinkertaisia että vaativampia kyselytilanteita, kuten taululiitoksia, suodatuksia ja aggregaatioita.

Aineisto sisältää kaupallisesti arkaluonteista tietoa, minkä vuoksi varsinaisia kyselytuloksia ei julkaista. Tästä syystä tutkimuksessa vertaillaan järjestelmän tuottamia SQL-kyselyitä eikä niiden palauttamia tietosisältöjä. Ratkaisu tukee myös analyysin johdonmukaisuutta, koska virheellisen kyselyn vaikutus lopputulokseen ei ole aina pääteltävissä pelkästään syntaksista, vaan riippuu osittain tietokannan sisällöstä.

6.3 Cortex Analyst ja semanttinen näkymä

Tutkimuksessa käytettävä järjestelmä on Snowflaken Cortex Analyst [39]. Järjestelmän tarkoituksena on mahdollistaa tietokantatiedon hyödyntäminen luonnollisen kielen kautta siten, että käyttäjän ei tarvitse tuntea SQL-kieltä tai tietokannan skeemaa yksityiskohtaisesti. Cortex Analyst muodostaa käyttäjän kysymyksestä SQL-kyselyn ja voi suorittaa sen tietokantaa vasten.

Cortex Analystin toiminta perustuu semanttiseen näkymään, jossa tietokannan taulut, kentät ja niiden merkitykset kuvataan YAML-muotoisena määrittelynä (ks. kuva 2). Semanttinen näkymä tarjoaa kielimallille rakenteellisen kuvauksen siitä, mitä tietoa tietokannassa on saatavilla ja miten eri taulut liittyvät toisiinsa. Näin järjestelmä pystyy muodostamaan syntaktisesti valideja SQL-kyselyitä ja kohdistamaan kyselyn oikeisiin tietokannan osiin. Semanttisen näkymän laatu on keskeinen osa järjestelmän toimivuutta, koska kielimallin tulkinta perustuu olennaisesti siihen, miten skeema on kuvattu.

Attribute	**Value**
PARTICIPANTS_ID	ea0a918f
IDENTIFIER	63c44469
QUEUEID	3b10a230
CALLERS	+123456789012
AGENTID	b7e632de
SUBJECT	Null
ADDRESS_TO	Null
ADDRESS_FROM	Null
CONTACT_DURATION	363
DIRECTION	inbound
DIRECTION_ORIGINAL	inbound
ANSWERED	1
QUEUE_DURATION	2
CALL_DURATION	316
WRAPUPID	dd91b10d
WRAPUP_DURATION	47
CONTACT_CLOSED_REASON	client
CONTACT_CONNECTED_DATETIME	2025-08-23 13:41:13.000
CONTACT_CLOSED_DATETIME	2025-08-23 13:47:28.000
CALL_ANSWERED_DATETIME	2025-08-23 13:41:25.000
CALL_DISCONNECTED_DATETIME	2025-08-23 13:46:41.000
MEDIA_TYPE	call
CONTACT_CONNECTED_DATE	2025-08-23 00:00:00.000

Taulukko 1: Esimerkki yhdestä tietokannan rivistä. Osa kentistä on anonymisoitu tietosuoja- ja luottamuksellisuussyistä.

Tutkimuksessa käytetty semanttinen näkymä rakennettiin tutkijan, IT-konsulttiyritys Knowitin ja yhden sen asiakkaan yhteistyönä osana hanketta, jossa pyritään hyödyntämään kielimalleja tietokannan käytön helpottamiseksi. Tutkija osallistui siten arvioitavan järjestelmän semanttisen kontekstin rakentamiseen sekä laati tutkimuksessa käytetyt kysymykset ja arvioi tuotetut SQL-kyselyt. Sovellusympäristön tuntemus tuki kysymysten laatimista ja kyselyiden arviointia, mutta kehittämis- ja arviointiroolien yhdistyminen on huomioitava tutkimuksen riippumattomuutta tarkasteltaessa.

Tässä tutkimuksessa käytetty semanttinen näkymä kuvailee kaikki kuvassa 1 esiintyvät taulut, niiden kentät ja suhteet toisiinsa sekä useita kentistä laskettuja

metriikoita. Kuvassa 2 on esitetty yksinkertaistettu versio käytetystä semanttisesta näkymästä.

Kokeiden aikana Cortex Analyst käytti taustallaan Anthropicin Claude Sonnet 4 -mallia [1]. Tutkimuksen tulokset kuvaavat ensisijaisesti juuri tämän järjestelmän ja mallin yhdistelmän toimintaa. Huhtikuussa 2026 Cortex Analyst voi käyttää uudempaa Claude Sonnet 4.6 -mallia [39]. Tuloksia ei siksi voida sellaisenaan yleistää kaikkiin kielimalleihin tai muihin luonnollisen kielen tietokantarajapintoihin.

6.4 Arviointiperusteet

Tuotettuja SQL-kyselyitä arvioidaan useiden toisiaan täydentävien kriteerien avulla. Ensinnäkin tarkastellaan, valitseeko järjestelmä oikeat taulut ja käyttääkö se oikeita liitoksia silloin, kun käyttäjän kysymys edellyttää useamman taulun yhdistämistä. Toiseksi arvioidaan suodatusehtoja, erityisesti `where`-lauseiden sisältöä, koska pienikin virhe niissä voi muuttaa kyselyn merkitystä olennaisesti. Lisäksi huomioidaan aggregaatiot, duplikaattien käsittely ja sarakevalinnat. Nämä kriteerit vastaavat taulukon 4 virhetyyppejä.

Syntaktista validiutta ei tarkastella erikseen, koska kaikki järjestelmän tuottamat kyselyt olivat syntaktisesti valideja.

Kukin vastaus luokitellaan yhteen seuraavista neljästä luokasta: *Oikein*, *Melkein oikein*, *Väärin* ja *Ei kyselyä*. Kysely luokitellaan luokkaan *Oikein*, jos sen perusteella saatava vastaus vastaisi esitettyyn luonnollisen kielen kysymykseen oikein ja huomioisi ne implisiittiset taustaoletukset, jotka ovat sovellusalueen näkökulmasta perusteltuja.

Tutkittava tietokanta sisältää puhelinkeskusdataa, joten erityisesti puhelun suuntaan liittyvät taustaoletukset ovat keskeisiä. Esimerkiksi palvelutasoa koskevan kysymyksen implisiittisenä taustaoletuksena pidetään sitä, että tarkastelu kohdistuu vain saapuviin kontakteihin. Oikein-luokan kysely huomioi tällaiset taustaoletukset.

Kysely luokitellaan luokkaan *Melkein oikein*, jos se tulkitsee käyttäjän tiedontarpeen oikein, mutta sisältää teknisen puutteen, joka voi vaikuttaa kyselyn suorittamiseen tai tuloksen oikeellisuuteen. Tällainen puute on esimerkiksi jakolaskun nimittäjästä puuttuva `NULLIF`-funktio, jolloin kysely voi johtaa virheeseen nimittäjän ollessa nolla. Myös aineistossa esiintyvien duplikaattien puutteellinen käsittely luetaan tähän luokkaan. Kyse on tällöin aineiston ominaisuuksien puutteellisesta huomioimisesta, ei tiedontarpeen virheellisestä tulkinnasta.

Kysely luokitellaan luokkaan *Väärin*, jos se ei vastaa kysymykseen merkityksellisellä tavalla. Esimerkiksi kysely, joka käyttää väärää suodatusta tai josta puuttuu olennainen taulu tai kenttä, luokitellaan vääräksi.

Jos malli ei tuota lainkaan SQL-kyselyä, vaan ilmoittaa, että kysymykseen ei voida vastata, vastaus luokitellaan luokkaan *Ei kyselyä*. Kaikkiin tutkimuksessa käytettyihin kysymyksiin on mahdollista vastata käytettävissä olevan aineiston ja semanttisen näkymän perusteella, joten tähän luokkaan kuuluvat vastaukset ovat virheellisiä. Ne on kuitenkin mielekästä erottaa luokasta *Väärin*, sillä vaikka ne antavat harhaanjohtavan kuvan siitä, mihin kysymyksiin tietokannan rakenteen ja semanttisen näkymän perusteella voidaan vastata, ne eivät tuota virheellisiä kyselytuloksia tietokannan sisältämistä tiedoista.

Samaan luonnollisen kielen kysymykseen voi olla useita oikeita SQL-vastauksia. Esimerkiksi kysymykseen *How many contacts did we handle in January 2026?* järjestelmä voi tuottaa kyselyn, joka palauttaa yhden kokonaisluvun, tai kyselyn, joka palauttaa kontaktien määrät kuukausittain, jolloin käyttäjä voi poimia vastauksen taulusta. Tässä tutkielmassa kysely tulkitaan oikeaksi, jos sen tuottama vastaus sisältää käyttäjän tiedontarpeen.

Oikeellisuuden lisäksi tarkastellaan kyselyiden vaihtelua tiedontarpeen tulkinnassa ja SQL-kyselyn toteutustavassa. *Tulkinnan* tasolla kyselyt katsotaan samoiksi, jos ne toteuttavat saman käyttäjän tiedontarpeen samoin rajauksin ja taustaoletuksin. Tämä ei edellytä SQL-kyselyiden palauttamien tulosrelaatioiden täydellistä yhtäsuuruutta. Tulosten esitysmuoto tai laajuus voi erota, kunhan käyttäjän tarvitsema tieto on saatavissa samoin rajauksin ja taustaoletuksin. Tulkinnallinen konsistenssi tarkoittaa tiedontarpeen tulkinnan säilymistä samana toistuvissa suorituksissa, eikä se edellytä saman SQL-toteutuksen käyttämistä.

Tässä tutkimuksessa tulkinnaltaan samat kyselyt voivat erota *toteutustavaltaan* kahdella tavalla. Ensinnäkin kyselyt voivat erota siinä, missä muodossa ja kuinka laajan tuloksen osana käyttäjän tarvitsema tieto palautetaan. Toiseksi kyselyt voivat hakea saman tiedon eri lähteistä. Tämä koskee tilanteita, joissa sama tieto on saatavilla useasta sarakkeesta tai taulusta, sekä tilannetta, jossa arvo voidaan laskea joko suoraan taulun kentistä tai käyttämällä semanttiseen näkymään määriteltyä metriikkaa.

Edellä esitetty esimerkki tammikuun kontaktimäärän palauttamisesta havainnollistaa tätä erottelua. Yhden kokonaisluvun palauttava kysely ja kuukausittaisen jakauman palauttava kysely edustavat samaa *tulkintaa*, jos molemmista saadaan tammikuun kontaktimäärä samoin rajauksin ja taustaoletuksin. Ne edustavat kuitenkin kahta eri *toteutustapaa*, koska käyttäjän tarvitsema tieto palautetaan eri laajuisen tuloksen osana. Jos nämä ovat ainoat havaitut vaihtoehdot, tulkintojen määrä on yksi ja toteutustapojen määrä kaksi.

Sama tiedontarpeen tulkinta ei takaa kyselyiden tulosten yhtäsuuruutta, sillä tulkinnan toteuttamisessa voi esiintyä teknisiä puutteita. Tällaiset puutteet arvioidaan oikeellisuuden luokittelussa, eivätkä ne tässä luokittelussa yksin muodosta uutta tulkintaa tai toteutustapaa.

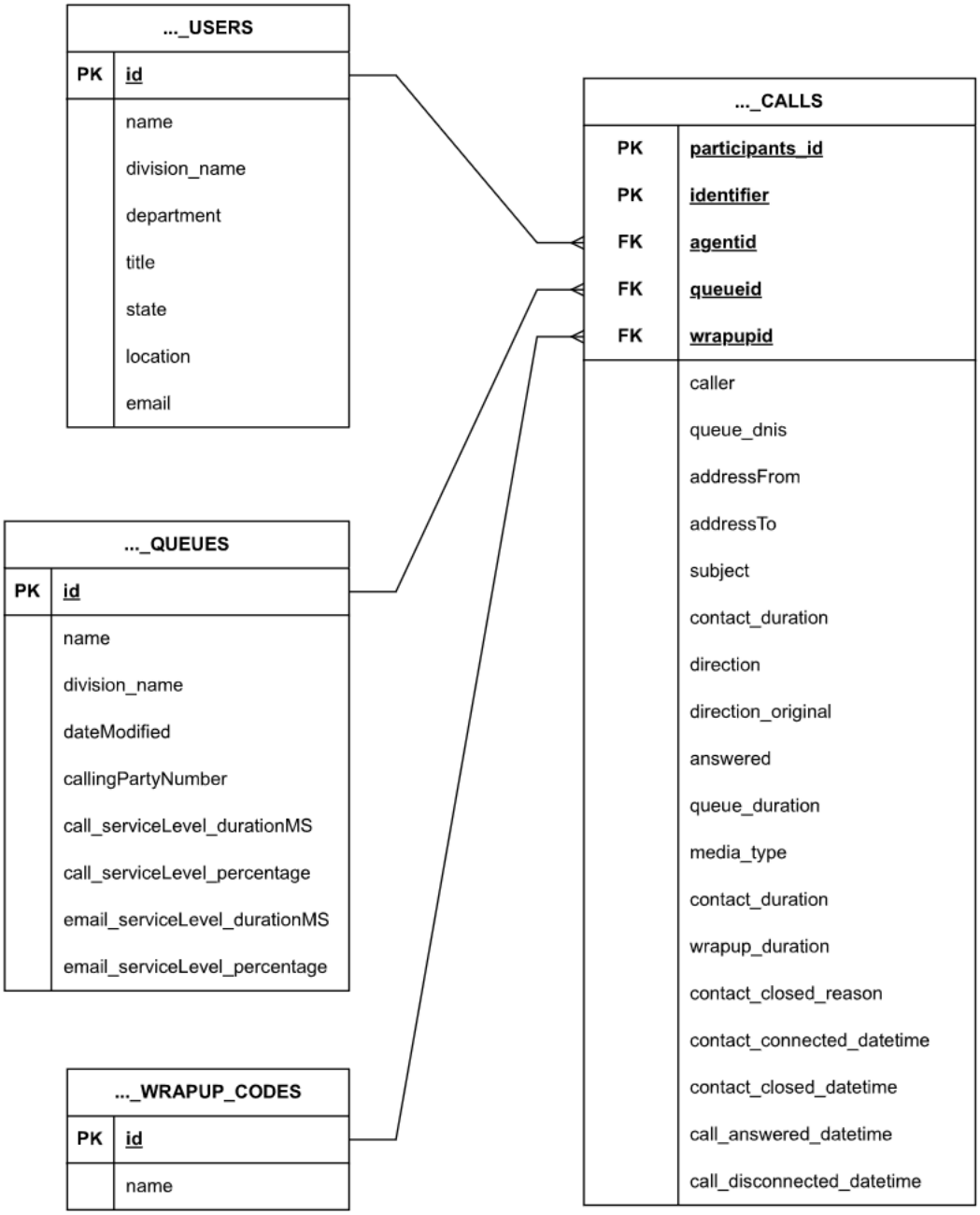
Pelkkiä syntaktisia eroja, kuten taulu- ja sarakealiasten nimiä, muotoilua tai suodatusehtojen järjestystä, ei lasketa tulkinta- tai toteutusvaihteluksi. Toteutuserot eivät itsessään kerro kyselyn oikeellisuudesta. Tekniset puutteet, kuten tarpeeton `distinct` tai puuttuva `NULLIF`, eivät muodosta uutta toteutustapaa, vaan ne huomioidaan oikeellisuuden luokittelussa.

Toteutustavat erotellaan kunkin tulkinnan sisällä. Tulosten taulukoissa sarake *Tulk.* ilmoittaa rivillä havaittujen erilaisten tulkintojen määrän ja sarake *Tot.* näiden tulkintojen sisällä tunnistettujen toteutustapojen yhteenlasketun määrän. Esimerkiksi kaksi tulkintaa, joilla kummallakin on yksi toteutustapa, tuottaa arvot *Tulk.* $= 2$ ja *Tot.* $= 2$, vaikka saman tulkinnan sisällä ei esiinny toteutusvaihtelua. Jos yhdellä tulkinnalla on kaksi toteutustapaa, vastaavat arvot ovat *Tulk.* $= 1$ ja *Tot.* $= 2$.

Ryhmän *Yht.*-rivi ilmoittaa koko kysymysryhmässä havaittujen erilaisten tulkintojen määrän ja niiden sisällä tunnistettujen toteutustapojen yhteenlasketun mää-

rän. Sama tulkinta ja siihen kuuluva sama toteutustapa lasketaan ryhmän yhteenvetoon vain kerran riippumatta siitä, kuinka monessa kysymysmuotoilussa tai suorituksessa ne esiintyvät.

Tuotettujen SQL-kyselyiden luokittelu tehtiin manuaalisesti vertaamalla niitä tutkijan tulkintaan siitä, millainen kysely vastaisi käyttäjän tiedontarvetta. Arvioinnin suoritti tutkija yksin, eikä luokittelua varmennettu riippumattomalla arvioijalla. Koska tutkija oli myös osallistunut semanttisen näkymän rakentamiseen ja laatinut kokeissa käytetyt kysymykset, arvioinnissa käytetty vertailutulkinta ei ollut riippumaton järjestelmän kehittämisestä ja kysymysten valinnasta. Edellä kuvatut arviointiperusteet tekevät luokittelun lähtökohdat näkyviksi, mutta eivät poista arvioijan tulkintaan liittyvän subjektiivisuuden mahdollisuutta. Useiden arvioijien välistä yhtenevyyttä ei tässä tutkimuksessa voitu arvioida.



Kuva 1: Tutkimuksessa käytettävän tietokannan rakenne. Taulujen nimet on lyhennetty luettavuuden vuoksi.

```yaml
name: SEM_EXAMPLE
description: |-
  Semantic model that models conversations data from Genesys
     Cloud contact center software.
tables:
  - name: CONV
    description: |-
      All Genesys conversations.
    base_table:
      database: DEV
      schema: DBT_OM
      table: FACT_GENESYS__CONVERSATIONS_STANDARD
    dimensions:
      - name: C_ADDRESS_FROM
        description: Source address for the interaction. Only
            present in email contacts..
        expr: conv.address_from
        data_type: VARCHAR(16777216)
    facts:
      - name: CALL_DURATION
        description: Duration of the actual call in seconds.
           The time the customer speaks to an agent
        expr: conv.call_duration
        data_type: NUMBER(38,0)
        access_modifier: public_access
    metrics:
      - name: AVERAGE_CALL_DURATION
        description: Average call duration in seconds
        expr: avg(conv.call_duration)
        access_modifier: public_access
    primary_key:
      columns:
        - PARTICIPANTS_ID
        - IDENTIFIER
```

Kuva 2: Esimerkki semanttisesta näkymästä.

7 Kokeet

Ensimmäisessä kokeessa tarkastellaan Cortex Analystin tulkinnallista konsistenssia eli sitä, säilyykö järjestelmän tulkinta käyttäjän tiedontarpeesta samana, kun sama luonnollisen kielen kysymys esitetään useita kertoja. Tästä erillään tarkastellaan toteutusvaihtelua eli sitä, millaisia SQL-toteutuksia järjestelmä muodostaa saman tulkinnan pohjalta.

Toisessa kokeessa tarkastellaan järjestelmän herkkyyttä kysymyksen sanamuodon muutoksille, joiden on tarkoitus säilyttää tiedontarpeen merkitys. Arvioinnissa erotetaan muutokset tiedontarpeen tulkinnassa muutoksista SQL-kyselyn toteutustavassa.

Tutkimukseen valittiin tarkoituksenmukaisesti viisi kysymystä, jotka edustavat erilaisia puhelinkeskusdatan analyysitehtäviä. Kysymykset laati tutkija sovellusympäristön analyysitarpeiden ja semanttisen näkymän rakenteen perusteella. Valinnassa pyrittiin sisällyttämään sekä valmiiden metriikoiden tukemia että useiden kenttien ja ehtojen yhdistämistä edellyttäviä tiedontarpeita.

Kysymysjoukkoon sisältyy yksinkertainen kontaktimäärän laskenta (A), vastausprosentin määrittäminen (B), ehdollinen prosenttilaskenta ja toimialan taustaoletusten huomioiminen (C), kontaktien ajallinen jakauma (D) sekä useita osatehtäviä yhdistävä takaisinsoittojen analyysi (E). Kysymykset A ja B tarkastelevat tilanteita, joissa semanttiseen näkymään on määritelty tiedontarvetta tukevia valmiita metriikoita. Kysymykset C–E puolestaan edellyttävät useiden kenttien, ehtojen tai laskentavaiheiden yhdistämistä.

Kysymysjoukko mahdollistaa erilaisten kyselytilanteiden havainnollistavan tarkastelun, mutta sen perusteella ei voida erottaa esimerkiksi tehtävän monimutkaisuuden ja valmiiden metriikoiden vaikutuksia järjestelmän suoriutumiseen.

Valinnan tavoitteena oli mahdollistaa erilaisten kyselytilanteiden yksityiskohtainen tarkastelu rajatussa pilottitutkimuksessa, ei muodostaa tilastollisesti edustavaa otosta kaikista mahdollisista käyttäjäkysymyksistä.

Konsistenssikoe Koska kielimallit ovat tilastollisia järjestelmiä, sama syöte ei välttämättä aina johda täsmälleen samaan tulosteeseen. Tietokantakyselyiden tapauksessa tämä on olennaista, koska käyttäjät odottavat järjestelmältä ennustettavaa ja toistettavaa toimintaa.

Konsistenssikokeessa sama luonnollisen kielen kysymys esitetään järjestelmälle 10 kertaa. Jokaisesta suorituksesta tallennetaan järjestelmän tuottama SQL-kysely, joita verrataan keskenään. Käytetyt kysymykset ovat seuraavat

A. *How many total contacts did we handle in January 2026?*

B. *What's our answer rate for calls in February 2026?*

C. *What percentage of contacts that were answered within our service level target were closed by the customer versus closed by the agent?*

D. *What was our peak contact hours by day of week? Show me the distribution of when contacts arrive throughout the day, broken down by hour and weekday.*

E. *What's the success rate of our callbacks? Show me how many callbacks resulted in a conversation versus no answer, and what the average time between the original inbound contact and the callback was.*

Määritellyssä semanttisessa näkymässä on kuvattu metriikoihin SQL-kyselyt, jotka vastaavat suoraan kysymyksiin A ja B. Kysymykset C-E ovat huomattavasti monimutkaisempia, ja niihin ei ole ennalta validoituja SQL-kyselyitä semanttisessa näkymässä. Kysymys D vaatii, että kentästä `CONTACT_CONNECTED_DATETIME` erotetaan kellonaika tunnin tarkkuudella, ja viikonpäivä pitää hakea kalenteritaulusta. Kysymys E on rakenteeltaan monitulkintainen. Erityisesti takaisinsoiton ja alkuperäisen sisääntulevan kontaktin välisen ajan keskiarvoa koskeva osa voidaan tulkita vaativan kahden eri rivin välistä vertailua, vaikka käytetyssä aineistossa vastaus voidaan muodostaa yhdeltä takaisinsoittoa kuvaavalta riviltä.

Herkkyyskoe Kokeen tavoitteena on arvioida, säilyykö Cortex Analystin tulkinta käyttäjän tiedontarpeesta samana kysymyksen sanamuodon muuttuessa. Tästä erillään tarkastellaan, tuottavatko eri muotoilut saman tulkinnan pohjalta vaihtoehtoisia SQL-toteutuksia.

Pelkkä toteutusvaihtelu ei merkitse epäjohdonmukaista tulkintaa. Jos sanamuodon muutoksiin liittyy tiedontarpeen tulkinnan muutoksia tai kyselyiden oikeellisuuden heikkenemistä, luonnollisen kielen rajapinnan toiminta on käyttäjän kannalta vaikeammin ennustettavaa. Tiedontarpeen tulkinnan säilyminen samana eri sanamuodoista huolimatta puolestaan tukee järjestelmän ennustettavuutta käytännön työssä.

Koe toteutetaan muodostamalla kustakin konsistenssikokeessa käytetystä kysymyksestä 4 vaihtoehtoista versiota, joissa pyritään säilyttämään kysymyksen merkitys, mutta muokkaamaan sanamuotoa. Variaatiot muodosti tutkija, eikä niiden merkitysvastaavuutta varmennettu riippumattomasti. Variaatioiden tarkoituksena ei ole ollut muodostaa formaalisti ekvivalentteja kysymyksiä vaan realistisia vaihtoehtoisia ilmaisutapoja samalle tiedontarpeelle.

Käytetyt kysymykset ovat seuraavat

A. Variaatioita kysymyksestä A:

    1. *What was the total number of contacts we handled in January 2026?*
    2. *How many contacts did we handle overall in January 2026?*
    3. *What is the total contact volume for January 2026?*
    4. *How many total customer contacts were handled during January 2026?*

B. Variaatioita kysymyksestä B:

    1. *What was our call answer rate in February 2026?*
    2. *What percentage of incoming calls were answered in February 2026?*
    3. *What percentage of calls were answered in February 2026?*
    4. *How did our call answer rate perform during February 2026?*

C. Variaatioita kysymyksestä C:

1. *Of the contacts answered within our service-level target, what share were closed by the customer compared to those closed by the agent?*
2. *For interactions resolved within the service level target, what percentage were customer-closed versus agent-closed?*
3. *Within our service level target, how is the closure split: what percent of contacts were closed by customers versus agents?*
4. *Among contacts answered within SLA, what proportion ended with customer closure versus agent closure?*

D. Variaatioita kysymyksestä D:

1. *When do contacts most frequently arrive during the week? Break this down by weekday and hour.*
2. *Show the hourly contact-arrival distribution for every day of the week, highlighting which hours were highest on each weekday.*
3. *What times during the day (hour-by-hour) drive the most contacts, split by weekday. Please summarize peak hours and the overall distribution.*
4. *Across the week, when do contacts arrive most often? Provide an hour-by-hour breakdown for each weekday and identify the peak contact hours.*

E. Variaatioita kysymyksestä E:

1. *What percentage of our callback attempts lead to an actual conversation? Please break down callbacks that reached a conversation vs. those with no answer, and calculate the average delay from the original inbound to the callback.*
2. *How effective are our callbacks? Show the success rate by counting callbacks that resulted in a conversation versus no answer, and report the average time-to-callback from the initial inbound contact.*
3. *For our callback campaign, what's the outcomes split and average response speed? Provide counts of conversations vs. no-answer callbacks, the resulting success rate, and the average elapsed time between inbound contact and callback.*
4. *Can you analyze callback performance for me: success rate (conversation vs no answer) with the number of callbacks in each outcome category, plus the average time lag from the inbound contact to when the callback occurred?*

Konsistenssikokeessa tehtiin 50 suoritusta ja vaihtoehtoisilla kysymysmuotoiluilla 200 suoritusta. Herkkyysanalyysissa alkuperäisten kysymysten vertailuaineistona käytettiin konsistenssikokeen suorituksia. Erillisiä suorituksia oli siten yhteensä 250. Jokainen yksittäinen suoritus tehtiin tyhjällä sessiolla. Testit suoritettiin 23.03.2026-28.03.2026 välisenä aikana käyttäen samaa semanttista mallia.

Kymmenen toistoa valittiin käytännöllisenä kompromissina, joka mahdollistaa vaihtelun havaitsemisen ilman kohtuuttoman suurta manuaalisen arvioinnin työmäärää.

8 Tulokset

Merkitään konsistenssikokeen kysymykset A-E ja herkkyyskokeen variaatiot A1-A4, B1-B4, C1-C4, D1-D4 ja E1-E4 listojen 7 ja 7 mukaisesti. Kokeiden tulokset on esitetty taulukoissa 2 ja 3. Taulukot kuvaavat kunkin kysymyksen osalta, kuinka monta kertaa järjestelmän tuottama SQL-kysely oli oikein, melkein oikein, virheellinen tai johti tilanteeseen, jossa SQL-kyselyä ei tuotettu lainkaan.

Kysymykset A ja D tuottivat konsistenssikokeessa kumpikin vain yhden tiedontarpeen tulkinnan ja yhden toteutustavan. Tuotetut SQL-kyselyt vastasivat käyttäjän tiedontarpeeseen oikein.

Kysymyksessä B havaittiin yksi tiedontarpeen tulkinta ja kaksi sen toteutustapaa. Järjestelmä joko käytti semanttisessa näkymässä määriteltyä metriikkaa `answer_percentage` tai laski vastaavan prosenttiluvun suoraan taulun kentistä. Tulkinnallista vaihtelua ei havaittu, vaan molemmat toteutustavat vastasivat samaan tiedontarpeeseen oikein.

Konsistenssikokeessa vain kysymys C tuotti sekä oikeita että virheellisiä SQL-kyselyitä. Järjestelmä tulkitsi kysymyksen väärin kolmessa suorituksessa, joissa asiakkaan ja agentin roolit sekoittuivat keskenään. Seitsemässä suorituksessa kymmenestä järjestelmä tulkitsi kysymyksen oikein ja tuotti tiedontarvetta vastaavan SQL-kyselyn. Saman tulkinnan sisällä ei havaittu toteutusvaihtelua.

Kysymys E ei tuottanut SQL-kyselyä yhdelläkään suorituskerralla, vaan järjestelmä ilmoitti toistuvasti ja virheellisesti, ettei käytettävissä oleva aineisto riittänyt kysymykseen vastaamiseen. Sen kohdalla havaittiin siten johdonmukainen kieltäytyminen kyselyn muodostamisesta, mutta SQL-kyselyiden tulkinta- tai toteutusvaihtelua ei voitu arvioida. Tämän vuoksi kysymyksen E tulkintojen ja toteutustapojen määrät on merkitty taulukkoon viivoilla. Kysymys E osoittaa, ettei vastauskäyttäytymisen toistettavuus yksin riitä kuvaamaan järjestelmän luotettavuutta, mikä heijastaa Text-to-SQL-evaluoinnissa tunnettua eroa toistettavuuden ja semanttisen virheettömyyden välillä [28].

Seuraavassa esitetään esimerkki kysymykseen E annetusta *Ei kyselyä* -luokkaan kuuluvasta vastauksesta

> *I'm sorry, but this question cannot be fully answered with the available data. While we can determine how many callbacks were answered versus not answered (using the c_answered field filtered for callback media type), the semantic model does not contain information about the original inbound contact that triggered a callback. There is no field linking a callback conversation back to its originating inbound contact, so we cannot calculate the time between the original inbound contact and the callback attempt.*

Järjestelmän vastaus perustuu virheelliseen tulkintaan käytettävissä olevasta tiedosta. Tutkimuksessa käytetyn tietomallin mukaan takaisinsoitto tunnistetaan rivistä, jolla `direction`-sarakkeen arvo on `outbound` ja `direction_original`-sarakkeen arvo on `inbound`. Näillä riveillä `c_contact_connected_datetime` ilmaisee alkuperäisen saapuvan soiton ajankohdan, ei takaisinsoiton alkamisajankohtaa. Alkuperäisen yhteydenoton aikaleima siis säilyy samalla rivillä, vaikka rivin kuvaaman puhelun

suunta on takaisinsoiton vuoksi ulospäin. Sarake `c_call_answered_datetime` puolestaan ilmaisee ajankohdan, jolloin takaisinsoiton vastaanottaja vastasi puheluun. Näiden aikaleimojen erotuksena voidaan siten laskea aika alkuperäisestä saapuvasta soitosta takaisinsoittoon vastaamiseen. Laskenta ei edellytä järjestelmän vastauksessa mainittua erillistä yhteyttä alkuperäisen kontaktin ja takaisinsoiton välillä, sillä molemmat tarvittavat aikaleimat sisältyvät samaan riviin.

Konsistenssikokeen 50 suorituksesta 37 tuotti oikean SQL-kyselyn, 10 ei tuottanut SQL-kyselyä lainkaan ja 3 tuotti virheellisen SQL-kyselyn. Konsistenssikokeessa enemmistö suorituksista tuotti oikeaksi arvioidun SQL-kyselyn, mutta oikeellisuus ja tulkinnallinen konsistenssi vaihtelivat viiden tutkitun tiedontarpeen välillä.

Tulokset viittaavat siihen, että tässä tutkimusasetelmassa havaitut ongelmat olivat usein toistettavia. Tämä on hyödyllinen havainto sekä käyttäjän että semanttisen näkymän kehittäjän näkökulmasta. Kysymyksessä C järjestelmä kuitenkin tuotti samassa asetelmassa sekä oikeita että virheellisiä SQL-kyselyitä, joten tiedontarpeen tulkinta ei säilynyt kaikissa suorituksissa samana.

Herkkyyskokeessa eniten tulkinnallista vaihtelua havaittiin kysymysryhmässä C, jossa tunnistettiin yhteensä neljä erilaista tiedontarpeen tulkintaa. Alkuperäinen kysymys C ja sen variaatiot tuottivat yhteensä 16 virheelliseksi luokiteltua SQL-kyselyä 50 suorituksessa.

Alkuperäisen kysymyksen C virheet liittyivät asiakkaan ja agentin roolien sekoittumiseen, kun taas sanamuotovariaatioissa korostuivat suuntasuodatuksen virheet. Näissä suodatusvirheissä järjestelmä käytti sarakkeen `direction` sijaan saraketta `direction_original` tai jätti suuntasuodatuksen kokonaan pois.

Kysymys C ja sen variaatiot koskevat palvelutasoa. Tutkimuksen arviointiperusteissa palvelutasoa koskevien kysymysten implisiittiseksi taustaoletukseksi määriteltiin, että tarkastelu kohdistuu vain saapuviin kontakteihin. Havainto viittaa siihen, että Cortex Analyst ei tässä tutkimusasetelmassa aina huomioinut tätä taustaoletusta johdonmukaisesti. Luonnollisen kielen kysymyksissä esiintyvät monitulkintaiset ilmaukset ja implisiittisen toimialatiedon päättely muodostavat tunnetun haasteen Text-to-SQL-malleille [11, 6].

Variaatiossa C1 järjestelmä tulkitsi kahdesti, että asiakkaan päättämien kontaktien osuus tuli laskea suhteessa asiakkaan tai agentin päättämiin kontakteihin eikä kaikkiin palvelutasotavoitteen sisällä vastattuihin kontakteihin. Tällöin laskennan nimittäjästä jäivät pois kontaktit, joiden päättymissyy oli jokin muu tai puuttui, mikä johti asiakkaan päättämien kontaktien osuuden yliarviointiin.

Kysymysryhmissä A ja B ei havaittu virheelliseksi luokiteltuja SQL-kyselyitä. Variaatiossa A4 järjestelmä kuitenkin käytti tarpeettomasti lauseketta `distinct(identifier)`, minkä vuoksi kyselyt luokiteltiin melkein oikeiksi. Kysymysryhmän B herkkyyskokeessa havaittiin toteutusvaihtelua, mutta tiedontarpeen tulkinta säilyi oikeana. Yksi toteutustapa käytti semanttiseen näkymään määriteltyä metriikkaa, ja toinen laski vastaavan arvon suoraan taulun kentistä. Kolmas käytti samaa laskentatapaa kuin toinen, mutta palautti vastausprosentin lisäksi niiden kontaktien määrän, jotka suljettiin ilman jonotusta.

Kysymys D ja sen variaatiot tuottivat oikean SQL-kyselyn 47 suorituksessa 50:stä. Kolmessa suorituksessa järjestelmä tuotti virheellisen SQL-kyselyn, josta puuttui `direction_original`-sarakkeeseen perustuva suodatus. Virheellisellä tul-

kinnalla oli yksi toteutustapa ja oikealla tulkinnalla kaksi. Oikean tulkinnan toteutustavoista toinen palautti vain asianmukaisesti rajatun tuloksen ja toinen sekä rajatun että rajaamattoman tuloksen.

Kysymys E ja sen variaatiot tuottivat 40 suorituksessa 50:stä tekstivastauksen, jonka mukaan kysymykseen ei voitu vastata. Variaatio E2 tuotti yhdeksän oikeaa SQL-kyselyä ja yhden melkein oikeaksi luokitellun kyselyn. Jälkimmäisessä duplikaattien poistaminen kohdistui `identifier`-sarakkeen sijaan koko riviin, jolloin saman tunnisteen sisältäviä rivejä saattoi jäädä tulokseen. Toteutusvaihtelua ei havaittu.

Variaation E2 sanamuoto ei viittaa yhtä vahvasti kahden rivin välisen vertailun tarpeeseen kuin kysymyksen E muut muotoilut. Tämä saattaa osaltaan selittää, miksi juuri E2 tuotti oikeita SQL-kyselyitä, kun taas muut muotoilut eivät tuottaneet SQL-kyselyitä lainkaan.

Kysymysryhmässä E oikeita SQL-kyselyitä tuotettiin harvemmin kuin muissa kysymysryhmissä. Ryhmässä yleiset kieltäytymiset kyselyn muodostamisesta selittävät *Ei kyselyä* -luokan suuren osuuden virhetyyppien jakaumassa. Taulukon 4 mukaan muista virhetyypeistä yleisimpiä olivat suodatusvirheet ja duplikaattien hallintavirheet. Kaikki suodatusvirheet liittyivät sarakkeisiin `direction` ja `direction_original`. Havainto antaa aihetta hypoteesille, että sarakkeiden merkitysten selkeämpi erottelu semanttisessa näkymässä voisi vähentää suodatusvirheitä.

Takaisinsoittojen tunnistamista voisi mahdollisesti tukea erillinen `is_callback`-sarake, jos sen määrittely vastaisi aineistossa käytettyä takaisinsoiton tunnistamisehtoa. Tämän muutoksen vaikutusta ei kuitenkaan testattu tässä tutkimuksessa.

On kuitenkin huomioitava, että juuri nämä sarakkeet nousivat esiin tässä tutkimusasetelmassa. Toisenlaisessa tietokannassa tai toisessa sovellusympäristössä ongelmalliset kentät voisivat olla erilaisia. Havainto nostaa esiin semanttisen näkymän mahdollisen merkityksen havaittujen virheiden selittämisessä ja niiden vähentämisessä.

Voidaan esittää hypoteesi, että semanttisen näkymän huolellinen suunnittelu, tietokannan metadatan rikastaminen ja mallin iteratiivinen ohjaaminen voisivat parantaa järjestelmän suorituskykyä. Hypoteesin testaaminen edellyttää kuitenkin lisätutkimusta laajemmilla aineistoilla.

Duplikaattien hallintavirheillä tarkoitetaan tässä virheitä, joissa `distinct`-avainsanaa on käytetty tarpeettomasti tai väärin. Tutkimusasetelmassa vain kysymystä A vastaavissa SQL-kyselyissä `distinct`-avainsanaa ei tarvita. Muiden kysymysten kohdalla duplikaattien hallinta tulisi yleensä kohdistaa `identifier`-sarakkeeseen. Duplikaattien hallintavirheitä esiintyi tutkimuksessa yhteensä 18 kertaa 250 suorituksen aikana.

Taululiitosvirheitä ei havaittu. Tämä saattaa johtua siitä, että tutkimusasetelmassa käytetty tietokanta sisälsi vain neljä taulua (ks. kuva 1).

Aggregaatio- ja sarakevalintavirheet olivat myös harvinaisia, mikä viittaa siihen, että tässä tutkimusasetelmassa järjestelmä tunnisti kysymyksistä yleensä oikein, mitä attribuutteja käyttäjä halusi hakea, vaikka se ei aina onnistunut tuottamaan oikeaa SQL-kyselyä niiden perusteella.

Taulukko 2: Konsistenssikokeen tulokset. *Tulk.* ilmoittaa erilaisten tiedontarpeen tulkintojen määrän ja *Tot.* kunkin tulkinnan sisällä tunnistettujen toteutustapojen yhteenlasketun määrän. *Tot.*-arvon suuruus ei yksin osoita saman tulkinnan sisäistä toteutusvaihtelua. Viiva tarkoittaa, ettei tulkinta- tai toteutusvaihtelua voitu arvioida, koska järjestelmä ei tuottanut SQL-kyselyitä.

Kysymys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Tulk.	Tot.
A	10	0	0	0	1	1
B	10	0	0	0	1	2
C	7	0	3	0	2	2
D	10	0	0	0	1	1
E	0	0	0	10	-	-

Taulukko 3: Herkkyyskokeen tulokset. *Tulk.* ilmoittaa erilaisten tiedontarpeen tulkintojen määrän ja *Tot.* kunkin tulkinnan sisällä tunnistettujen toteutustapojen yhteenlasketun määrän. *Tot.*-arvon suuruus ei yksin osoita saman tulkinnan sisäistä toteutusvaihtelua. Ryhmäkohtaisilla *Yht.*-riveillä sama tulkinta ja siihen kuuluva sama toteutustapa lasketaan vain kerran riippumatta siitä, kuinka monessa kysymysmuotoilussa tai suorituksessa ne esiintyvät. Viiva tarkoittaa, ettei tulkinta- tai toteutusvaihtelua voitu arvioida, koska järjestelmä ei tuottanut SQL-kyselyitä.

Kysymys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Tulk.	Tot.
A	10	0	0	0	1	1
A1	10	0	0	0	1	1
A2	10	0	0	0	1	1
A3	10	0	0	0	1	1
A4	0	10	0	0	1	1
A yht.	40	10	0	0	1	1
B	10	0	0	0	1	2
B1	10	0	0	0	1	1
B2	10	0	0	0	1	1
B3	10	0	0	0	1	3
B4	10	0	0	0	1	2
B yht.	50	0	0	0	1	3
C	7	0	3	0	2	2
C1	8	0	2	0	2	2
C2	10	0	0	0	1	1
C3	0	0	10	0	1	1
C4	2	7	1	0	2	2

Taulukko 3 jatkuu

Kysymys	Oikein	Melkein oikein	Väärin	Ei kyselyä	Tulk.	Tot.
C yht.	27	7	16	0	4	4
D	10	0	0	0	1	1
D1	10	0	0	0	1	1
D2	10	0	0	0	1	1
D3	10	0	0	0	1	2
D4	7	0	3	0	2	2
D yht.	47	0	3	0	2	3
E	0	0	0	10	-	-
E1	0	0	0	10	-	-
E2	9	1	0	0	1	1
E3	0	0	0	10	-	-
E4	0	0	0	10	-	-
E yht.	9	1	0	40	1	1

Taulukko 4: Esiintyneiden virhetyyppien määrät kaikissa kokeissa. Rivit kuvaavat, kuinka monta kertaa kukin virhetyyppi esiintyi kussakin kysymyksessä tai sen variaatioissa. Jos yhdessä SQL-kyselyssä esiintyy useampi virhetyyppi, kysely lasketaan jokaiseen virhetyyppiin erikseen.

Kysymys	A	B	C	D	E	Yhteensä
Aggregaatio	0	0	2	0	0	2
Duplikaattien hallinta	10	0	7	0	1	18
Ei kyselyä	0	0	0	0	40	40
Sarakevalinta	0	0	3	0	0	3
Suodatus	0	0	11	3	0	14
Taululiitos	0	0	0	0	0	0

9 Rajoitukset

Tutkimukseen liittyy useita rajoituksia, jotka on huomioitava tuloksia tulkittaessa.

Ensinnäkin tutkimus kohdistuu yhteen järjestelmään, Snowflaken Cortex Analystiin, jonka taustalla toimii Anthropicin Claude Sonnet 4 -malli. Tulokset kuvaavat siten yhtä tiettyä toteutusta eivätkä ole suoraan yleistettävissä muihin kielimalleihin, SQL-avustajiin tai luonnollisen kielen analytiikkatyökaluihin.

Toiseksi tutkimus perustuu yhteen sovellusalueeseen eli puhelinkeskusdataan. Tämän tietokannan rakenne, sanasto ja analyysitarpeet voivat poiketa merkittävästi muiden toimialojen vastaavista ympäristöistä. On mahdollista, että järjestelmä suoriutuisi eri tavoin esimerkiksi talousdatan, lokidatan tai asiakastietojen yhteydessä.

Kolmanneksi tutkimuksessa tarkastellaan mallin tuottamia SQL-kyselyitä eikä kyselyiden palauttamia varsinaisia vastauksia. Tämä rajaus on perusteltu aineiston luottamuksellisuuden vuoksi, mutta samalla se tarkoittaa, että tutkimus ei mittaa suoraan liiketoiminnallisen lopputuloksen oikeellisuutta. Kysely voi olla rakenteeltaan puutteellinen tai vaihtoehtoisesti syntaktisesti erilainen mutta käytännössä toimiva, eikä tätä eroa voida aina arvioida täysin ilman varsinaista tulosaineistoa.

Neljänneksi semanttisen näkymän laatu vaikuttaa suoraan siihen, miten hyvin Cortex Analyst kykenee muodostamaan kyselyitä. Jos näkymässä on epäselvyyksiä, puutteita tai monitulkintaisia kuvauksia, osa havaituista ongelmista voi johtua itse määrittelystä eikä pelkästään kielimallin rajoituksista. Tämän vuoksi tutkimus arvioi samalla myös sitä, kuinka hyvin kyseinen semanttinen näkymä tukee luonnollisen kielen ja tietokantarakenteen välistä tulkintaa.

Viidenneksi kielimallien toimintaa luonnehtii tietty epädeterministisyys, minkä vuoksi tulokset voivat vaihdella eri suorituskerroilla tai järjestelmäversioiden välillä. Lisäksi kaupalliset kielimallipohjaiset järjestelmät voivat muuttua ajan myötä ilman, että kaikki muutokset ovat käyttäjälle näkyviä. Tämän vuoksi tutkimuksen havainnot kuvaavat järjestelmän toimintaa tiettynä ajankohtana ja tietyssä konfiguraatiossa eivätkä välttämättä pysyvää ominaisuutta [8].

Kuudenneksi herkkyyskokeen kysymysvariaatiot muodosti tutkija, eikä niiden merkitysvastaavuutta varmennettu riippumattomasti. Osa muotoiluista (esim. C2) voi muuttaa tai hämärtää alkuperäistä tiedontarvetta, joten havaittua vaihtelua ei voida tulkita pelkästään merkitystä säilyttävien muotoilumuutosten vaikutukseksi. Toisaalta käyttäjät muotoilevat kysymyksiä käytännössä epätarkasti, joten tällaiset variaatiot voivat kuvata myös realistista käyttöä.

Seitsemänneksi tutkimuksessa tarkasteltiin vain viittä tiedontarvetta. Vaikka kysymykset edustavat erilaisia puhelinkeskusdatan analyysitehtäviä, ne kattavat vain rajallisen osan mahdollisista käyttötilanteista. Kysymysten toistaminen ja sanamuotovariaatiot mahdollistavat valittujen tiedontarpeiden sisäisen vaihtelun tarkastelun, mutta eivät laajenna tutkimuksen kattavuutta uusiin analyysitehtäviin. Tutkimus on tämän vuoksi luonteeltaan eksploratiivinen pilottitutkimus, jonka havainnot kuvaavat järjestelmän toimintaa valituissa kyselytilanteissa ja tarjoavat lähtökohtia laajemmalle jatkotutkimukselle.

Kahdeksanneksi tutkija osallistui semanttisen näkymän rakentamiseen, laati kokeissa käytetyt kysymykset ja arvioi tuotetut SQL-kyselyt. Tämä sovellusympäristön

tuntemus tuki aineiston tulkintaa, mutta roolien yhdistyminen voi aiheuttaa vinoumaa sekä kysymysten valintaan että vastausten arviointiin. Kysymysjoukko voi esimerkiksi painottua tutkijalle tuttuihin analyysitarpeisiin tai semanttisen näkymän tunnettuihin ominaisuuksiin. Lisäksi tutkija voi pitää joitakin toimialaan liittyviä taustaoletuksia itsestään selvinä, vaikka ne eivät ilmene yksiselitteisesti kysymyksen sanamuodosta tai semanttisesta näkymästä. Arviointi perustuu siten tutkijan perusteltuun mutta ei riippumattomasti varmennettuun tulkintaan käyttäjän tiedontarpeesta. Koska arvioinnin suoritti yksi henkilö, arvioijien välistä yhtenevyyttä ei voitu tarkastella.

10 Johtopäätökset

Tämän eksploratiivisen pilottitutkimuksen tavoitteena oli tarkastella Snowflaken Cortex Analystin toimintaa viidessä puhelinkeskusdataan liittyvässä kyselytilanteessa. Tutkimus tuottaa tapauskohtaisia havaintoja SQL-kyselyiden oikeellisuudesta, tulkinnallisesta konsistenssista ja kysymysmuotoilujen vaikutuksista, mutta ei muodosta kattavaa arviota järjestelmän soveltuvuudesta luonnollisen kielen tietokantarajapinnaksi.

Tutkituissa kyselytilanteissa Cortex Analyst tuotti useissa suorituksissa oikeellisia SQL-kyselyitä ja säilytti tiedontarpeen tulkinnan johdonmukaisena. Kysymykset A ja B, joiden tiedontarpeita tukivat semanttiseen näkymään määritellyt valmiit metriikat, tuottivat oikeita ja tulkinnaltaan vakaita kyselyitä.

Tulkinnallinen konsistenssi ei kuitenkaan edellyttänyt saman toteutustavan käyttämistä. Esimerkiksi kysymyksessä B järjestelmä hyödynsi joko semanttiseen näkymään määriteltyä metriikkaa tai laski vastaavan arvon taulun kentistä, mutta tiedontarpeen tulkinta säilyi samana. Nämä havainnot antavat alustavaa näyttöä Cortex Analystin käyttökelpoisuudesta valituissa kyselytilanteissa, mutta niiden perusteella ei voida määrittää, millaisissa tehtävissä järjestelmä toimii yleisesti luotettavasti.

Tutkimuksessa havaittiin myös virheellisiä kyselyitä ja perusteettomia kieltäytymisiä kyselyn muodostamisesta. Havaitut ongelmat liittyivät implisiittisten taustaoletusten huomioimiseen, suodatukseen, duplikaattien hallintaan ja monitulkintaisiin kysymysmuotoiluihin. Nämä havainnot ovat linjassa aiemman Text-to-SQL-tutkimuksen kanssa, jossa vastaavien monimutkaisten ehtojen ja taustatietojen yhdistämisen on todettu olevan kielimalleille huomattava haaste [19, 49].

Herkkyyskokeessa kysymysten eri muotoilut johtivat sekä saman tulkinnan vaihtoehtoisiin SQL-toteutuksiin että tiedontarpeen tulkinnan muutoksiin. Pelkkä toteutusvaihtelu ei merkinnyt virheellistä tai epäjohdonmukaista tulkintaa. Sen sijaan tulkinnan muutoksiin liittyi virheellisiä kyselyitä, mikä heikentää luonnollisen kielen rajapinnan ennustettavuutta käyttäjän näkökulmasta. Havainto on yhdenmukainen kielimallien syöteherkkyyttä koskevan tutkimuksen kanssa [7]. Koska kysymysvariaatioiden merkitysvastaavuutta ei varmennettu riippumattomasti, havaittuja tulkintaeroja ei kuitenkaan voida selittää yksinomaan merkitystä säilyttävien sanamuodon muutosten vaikutuksella.

Tutkimuksessa yleisimmin havaitut suodatusvirheet liittyivät sarakkeiden `direction` ja `direction_original` käyttöön. Tämä antaa aihetta hypoteesille, että näiden kenttien merkitysten selkeämpi erottelu semanttisessa näkymässä voisi parantaa järjestelmän kykyä muodostaa oikeellisia SQL-kyselyitä. Laajemmin voidaan olettaa, että semanttisen näkymän huolellinen suunnittelu, yksiselitteinen nimeäminen ja iteratiivinen kehittäminen voisivat tukea luonnollisen kielen kyselyjärjestelmän luotettavuutta. Tutkimuksessa ei kuitenkaan vertailtu erilaisia semanttisia näkymiä, joten niiden vaikutusta järjestelmän suorituskykyyn ei voida näiden tulosten perusteella osoittaa.

Tutkimuksen perusteella tulkinnallista konsistenssia ei voida pitää yksinään riittävänä laadun mittarina. Järjestelmä saattoi toistaa saman virheellisen tulkinnan useilla suorituskerroilla. Lisäksi kysymyksessä E järjestelmä kieltäytyi johdonmukaisesti SQL-kyselyn muodostamisesta, vaikka kysymykseen voitiin vastata käytet-

tävissä olevan aineiston perusteella. Tämä osoittaa, ettei myöskään vastauskäyttäytymisen toistuvuus takaa oikeellisuutta. Toisaalta samaan kysymykseen tuotetut erilaiset SQL-toteutukset saattoivat perustua samaan oikeaan tulkintaan, jolloin toteutusvaihtelu ei itsessään merkinnyt luotettavuuden heikkenemistä. Luonnollisen kielen tietokantarajapintojen arvioinnissa on siten tarkasteltava oikeellisuutta ja tulkinnallista konsistenssia sekä raportoitava toteutusvaihtelu niistä erillään.

Pilottitutkimuksen havainnot ovat yhdenmukaisia aiemman kirjallisuuden kanssa, jossa luonnollisen kielen käyttöä tietokantojen rajapintana pidetään lupaavana mutta edelleen epävarmana lähestymistapana [21, 10].

Valituissa kyselytilanteissa Cortex Analyst tuotti sekä oikeiksi arvioituja SQL-kyselyitä että virheellisiksi arvioituja kyselyitä ja perusteettomia kieltäytymisiä kyselyn muodostamisesta. Havaintojen perusteella erityisesti implisiittisiä oletuksia, monitulkintaisuutta tai useita päättelyvaiheita sisältävissä tutkituissa kyselytilanteissa järjestelmän tuottamien kyselyiden oikeellisuus on syytä varmistaa.

Tutkielman tulokset viittaavat siihen, että käytännön kehitystyössä huomio kannattaa kohdistaa paitsi mallin suorituskykyyn myös siihen, miten järjestelmälle tarjottu semanttinen konteksti on rakennettu. Keskeinen jatkotutkimuksen tarve on laajentaa tarkastelua useampiin tiedontarpeisiin. Kysymysjoukko tulisi muodostaa ennalta määriteltyjen tehtävätyyppien tai todellisten käyttäjäkysymysten perusteella, jotta voitaisiin arvioida, toistuvatko tässä pilottitutkimuksessa havaitut ilmiöt laajemmassa aineistossa.

Tulevassa tutkimuksessa olisi hyödyllistä vertailla useita erilaisia semanttisia näkymiä, testata järjestelmää muissa sovellusympäristöissä sekä tarkastella tuotettujen SQL-kyselyiden lisäksi myös niiden palauttamien vastausten oikeellisuutta. Lisäksi useamman arvioijan käyttö voisi parantaa luokittelun reliabiliteettia ja vahvistaa tulosten luotettavuutta.

Tämä pilottitutkimus osoittaa, että Cortex Analyst voi tuottaa tiedontarvetta vastaavia SQL-kyselyitä valituissa puhelinkeskusdatan analyysitehtävissä, mutta oikeellisuus ei säilynyt kaikissa kysymysmuotoiluissa ja suorituksissa. Tulokset puoltavat tuotettujen kyselyiden oikeellisuuden varmistamista tässä sovellusympäristössä. Järjestelmän laajemman käyttökelpoisuuden arviointi edellyttää monipuolisempaa kysymysjoukkoa ja tuotettujen vastausten oikeellisuuden tarkastelua.

Viitteet

[1] Anthropic. System card: Claude Sonnet 4. Technical report, Anthropic, March 2026.

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[3] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. Neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.

[4] Léon Bottou. Online learning and stochastic approximations. In David Saad, editor, *Online Learning and Neural Networks*, pages 9–42. Cambridge University Press, Cambridge, UK, 1998. Revised version, May 2018. Available at `https://leon.bottou.org/publications/pdf/online-1998.pdf`.

[5] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.

[6] Shuaichen Chang, Jun Wang, Mingwen Dong, Lin Pan, Henghui Zhu, Alexander Hanbo Li, Wuwei Lan, Sheng Zhang, Jiarong Jiang, Xiang Deng, Kewei Tu, and Zhiguo Wang. Dr.Spider: A diagnostic evaluation benchmark towards robust text-to-SQL semantic parsing. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[7] Anwoy Chatterjee, H S V N S Kowndinya Renduchintala, Sumit Bhatia, and Tanmoy Chakraborty. POSIX: A prompt sensitivity index for large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 14550–14565, 2024.

[8] Lingjiao Chen, Matei Zaharia, and James Zou. How is ChatGPT's behavior changing over time? *Harvard Data Science Review*, (Special Issue 5), 2024.

[9] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? an analysis of BERT's attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 277–286. Association for Computational Linguistics, 2019.

[10] Naihao Deng, Yulong Chen, and Yue Zhang. Recent advances in text-to-SQL: A survey of what we have and what we expect. In *Proceedings of the 29th International Conference on Computational Linguistics (COLING)*, pages 2166–2187, 2022.

[11] Xiang Deng, Ahmed Hassan Awadallah, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. Structure-grounded pretraining for text-to-SQL. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 1337–1350, 2021.

[12] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.

[13] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[15] Zijin Hong, Zheng Yuan, Qinggang Zhang, et al. Next-generation database interfaces: A survey of LLM-based text-to-SQL. *arXiv preprint arXiv:2406.08426*, 2024.

[16] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. Learning deep structured semantic models for web search using clickthrough data. In *Proceedings of the 22nd ACM International Conference on Information & Knowledge Management (CIKM '13)*, pages 2333–2338, New York, NY, USA, 2013. Association for Computing Machinery.

[17] Dan Jurafsky and James H. Martin. *Speech and Language Processing*. Prentice Hall, 3rd ed. draft edition, 2026.

[18] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2015.

[19] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS 2023)*, volume 36, 2024.

[20] Seppo Linnainmaa. The representation of the cumulative rounding error of an algorithm as a taylor expansion of the local rounding errors. *Master's Thesis, University of Helsinki*, 1970.

[21] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. A survey of text-to-SQL in the era of LLMs: Where are we, and where are we going? *arXiv preprint arXiv:2408.05109*, 2024.

[22] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2019.

[23] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[24] Sewon Min, Julian Michael, Hannaneh Hajishirzi, and Luke Zettlemoyer. AmbigQA: Answering ambiguous open-domain questions. In *Proceedings of*

the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 5783–5797, 2020.

[25] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969.

[26] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, 2 edition, 2006.

[27] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 27730–27744, 2022.

[28] Mohammadreza Pourreza and Davood Rafiei. ETM: Modern insights into perspective on text-to-SQL evaluation in the age of large language models. *Future Internet*, 17(8):325, 2025.

[29] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. 2018.

[30] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond. *arXiv preprint arXiv:1904.09287*, 2018.

[31] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.

[32] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[33] Frank Rosenblatt. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan Books, Washington, DC, 1962.

[34] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[35] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.

[36] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3–4):379–423, 623–656, 1948.

[37] Claude E. Shannon. Prediction and entropy of printed english. *Bell System Technical Journal*, 30(1):50–64, 1951.

[38] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4596–4604. PMLR, 2018.

[39] Snowflake Inc. Cortex analyst. `https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst`, 2026. Viitattu: 22.4.2026.

[40] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[41] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *International Conference on Machine Learning*, pages 1139–1147. PMLR, 2013.

[42] Tijmen Tieleman and Geoffrey Hinton. Lecture 6a - neural networks for machine learning. *COURSERA: Neural Networks for Machine Learning*, 2012. Lecture notes / Coursera course material.

[43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

[44] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Radford, Uri Alon, and Adam Poliak. RAT-SQL: Addressing schema linking and extreme feature variations in natural language to SQL parsing. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7524–7537, Online, July 2020. Association for Computational Linguistics.

[45] Paul John Werbos. *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD thesis, Harvard University, 1974.

[46] Ashia C. Wilson, Rebecca Roelofs, Mitchell Stern, Nati Srebro, and Benjamin Recht. The marginal value of adaptive gradient methods in machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 4148–4158, 2017.

[47] Ruibin Xiong, Yichun Yang, Di He, Kaihe Zheng, Shuxin Zheng, Chenglin Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 10524–10533. PMLR, 2020.

[48] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[49] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921, 2018.

[50] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[51] Ruiqi Zhong, Tao Chou, Caiming Xiong, and Richard Socher. Semantic evaluation for text-to-SQL with distilled test suites. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 396–411, Online, November 2020. Association for Computational Linguistics.